The Complete Guide to Generative AI, RAG, and Agentic AI in Python

From Novice to Professional

Last Updated: May 2026 | **Estimated Reading Time:** 90+ minutes | **Skill Level:** Beginner → Professional

---

Table of Contents

1. Introduction to Generative AI
2. Setting Up Your Development Environment
3. Understanding Large Language Models (LLMs)
4. Working with LLM APIs
5. Local LLMs with HuggingFace
6. Retrieval-Augmented Generation (RAG)
7. Agentic AI Patterns and Frameworks
8. Advanced Topics and Best Practices
9. Capstone: Full-Stack AI Application

---

1. Introduction to Generative AI

1.1 What Is Generative AI?

Generative AI is a category of artificial intelligence systems that can **create new, original content** — text, images, code, music, video, and more — by learning patterns from massive datasets. Unlike traditional AI systems that classify, predict, or detect, generative models produce novel outputs that didn't exist before.

The field has undergone explosive growth since 2022, driven by breakthroughs in:

- **Large Language Models (LLMs):** GPT-4, Claude, Gemini, Llama — models that understand and generate human-like text
- **Image Generation:** DALL·E, Midjourney, Stable Diffusion — creating images from text descriptions
- **Code Generation:** GitHub Copilot, Cursor — AI-powered software development
- **Multimodal Models:** Systems that process and generate across text, image, audio, and video simultaneously

1.2 Why Learn Generative AI in Python?

Python is the **lingua franca** of AI development. Here's why:

Factor	Why Python Wins
Ecosystem	PyTorch, TensorFlow, HuggingFace, LangChain — all Python-first
API SDKs	OpenAI, Anthropic, Google all provide official Python clients
Community	Largest AI developer community; most tutorials, examples, and Stack Overflow answers
Simplicity	Clean syntax lets you focus on AI concepts, not language gymnastics
Data Science	Seamless integration with pandas, numpy, matplotlib for data work

1.3 What You'll Build in This Guide

By the end of this guide, you will have built:

1. **A multi-provider chatbot** that works with OpenAI, Anthropic, and local models
2. **A local text generation pipeline** using HuggingFace Transformers
3. **A RAG-powered document Q&A system** with vector search
4. **An autonomous AI agent** that can search the web, execute code, and write reports
5. **A full-stack AI application** combining all concepts

1.4 Prerequisites

- Basic Python knowledge (variables, functions, classes, f-strings)
- A computer with at least 8GB RAM (16GB+ recommended for local models)
- An internet connection for API access
- Curiosity and willingness to experiment

---

2. Setting Up Your Development Environment

A professional AI development environment eliminates friction so you can focus on building. We'll set up **VS Code** as our editor and **Pixi** as our package manager.

2.1 Installing Visual Studio Code

VS Code is the most popular editor for AI/ML development, offering excellent Python support, integrated terminals, and a rich extension ecosystem.

Installation:

1. Download VS Code from https://code.visualstudio.com (https://code.visualstudio.com)
2. Install for your operating system (Windows, macOS, or Linux)
3. Launch VS Code

Essential Extensions — Install These First:

Open the Extensions panel (`Ctrl+Shift+X` / `Cmd+Shift+X`) and install:

Extension	Purpose
Python (Microsoft)	Core Python support: IntelliSense, linting, debugging
Pylance (Microsoft)	Fast, feature-rich Python language server
Jupyter (Microsoft)	Run Jupyter notebooks directly inside VS Code
Pixi (prefix.dev)	Pixi environment integration for VS Code
GitHub Copilot (optional)	AI-powered code completion
Even Better TOML	Syntax highlighting for `pixi.toml` files

Configure Python Settings:

Open Settings (`Ctrl+,`) and set these key options:

```
{
    "python.analysis.typeCheckingMode": "basic",
    "python.analysis.autoImportCompletions": true,
    "editor.formatOnSave": true,
    "python.formatting.provider": "black",
    "[python]": {
        "editor.defaultFormatter": "ms-python.black-formatter",
        "editor.tabSize": 4
    }
}
```

2.2 Introduction to Pixi — Modern Python Package Management

Pixi is a next-generation, cross-platform package manager built on the conda ecosystem. It solves the biggest pain point in Python development: **dependency management**.

Why Pixi over pip/venv/conda?

Feature	pip + venv	conda	Pixi
Cross-platform lock-file	❌	❌	✅ `pixi.lock`
Speed	Medium	Slow	**Very Fast** (Rust)
Project-based	Manual	Manual	✅ `pixi.toml`
Reproducible builds	❌	Partial	✅ Exact
GPU package support	Limited	✅	✅
Task runner built-in	❌	❌	✅

Install Pixi:

```
# macOS / Linux
curl -fsSL https://pixi.sh/install.sh | sh

# Windows (PowerShell)
powershell -ExecutionPolicy ByPass -c "irm https://pixi.sh/install.ps1 | iex"

# Verify installation
pixi --version
```

2.3 Creating Your First AI Project with Pixi

Let's create a properly structured project:

```
# Create and enter project directory
pixi init genai-learning
cd genai-learning
```

This creates a `pixi.toml` file — the heart of your project configuration. Let's customize it:

```toml
# pixi.toml — Project configuration
[project]
name = "genai-learning"
version = "0.1.0"
description = "Learning Generative AI, RAG, and Agentic AI in Python"
channels = ["conda-forge"]
platforms = ["linux-64", "osx-arm64", "osx-64", "win-64"]

[dependencies]
python = ">=3.11,<3.13"

# Core AI libraries
openai = ">=1.40"
anthropic = ">=0.34"
transformers = ">=4.44"
torch = ">=2.4"
tokenizers = ">=0.19"

# RAG libraries
langchain = ">=0.3"
langchain-openai = ">=0.2"
langchain-community = ">=0.3"
chromadb = ">=0.5"
sentence-transformers = ">=3.0"

# Data and utilities
python-dotenv = ">=1.0"
rich = ">=13.0"
pandas = ">=2.2"
numpy = ">=1.26"

# Web and document loading
beautifulsoup4 = ">=4.12"
requests = ">=2.32"
pypdf = ">=4.0"

[tasks]
chat = "python src/chatbot.py"
rag = "python src/rag_app.py"
agent = "python src/agent.py"
test = "python -m pytest tests/"

[feature.dev.dependencies]
pytest = ">=8.0"
ipykernel = ">=6.29"
black = ">=24.0"
ruff = ">=0.5"
```

Install all dependencies:

```
pixi install
```

Create the project structure:

```
mkdir -p src tests data
touch src/__init__.py
touch src/chatbot.py src/rag_app.py src/agent.py
touch .env
```

Set up your API keys in `.env`:

```
# .env — NEVER commit this file to git!
OPENAI_API_KEY=sk-your-openai-key-here
ANTHROPIC_API_KEY=sk-ant-your-anthropic-key-here
```

Add `.env` to `.gitignore`:

```
echo ".env" >> .gitignore
echo "__pycache__/" >> .gitignore
echo ".pixi/" >> .gitignore
```

2.4 Verifying Your Environment

Create a test script to verify everything works:

```python
# src/verify_setup.py
"""Verify that the development environment is properly configured."""

import sys
from rich.console import Console
from rich.table import Table

console = Console()

def check_imports():
    """Check that all required packages are importable."""
    packages = {
        "openai": "OpenAI SDK",
        "anthropic": "Anthropic SDK",
        "transformers": "HuggingFace Transformers",
        "torch": "PyTorch",
        "langchain": "LangChain",
        "chromadb": "ChromaDB",
        "sentence_transformers": "Sentence Transformers",
        "dotenv": "python-dotenv",
        "pandas": "Pandas",
        "numpy": "NumPy",
        "bs4": "BeautifulSoup4",
    }

    table = Table(title="Environment Check", show_header=True)
    table.add_column("Package", style="cyan")
    table.add_column("Description", style="white")
    table.add_column("Version", style="green")
    table.add_column("Status", style="bold")

    all_ok = True
    for package, description in packages.items():
        try:
            mod = __import__(package)
            version = getattr(mod, "__version__", "N/A")
            table.add_row(package, description, version, "✅ OK")
        except ImportError as e:
            table.add_row(package, description, "—", f"❌ {e}")
            all_ok = False

    console.print(table)

    if all_ok:
        console.print("\n[bold green]✅ All packages installed correctly![/bold
green]")
    else:
        console.print("\n[bold red]❌ Some packages are missing. Run: pixi install[/
bold red]")

    return all_ok

def check_api_keys():
    """Check that API keys are configured."""
    import os
    from dotenv import load_dotenv
    load_dotenv()

    console.print("\n[bold]API Key Check:[/bold]")
    keys = {
        "OPENAI_API_KEY": os.getenv("OPENAI_API_KEY"),
        "ANTHROPIC_API_KEY": os.getenv("ANTHROPIC_API_KEY"),
```

```
    }
    for name, value in keys.items():
        if value and not value.startswith("sk-your"):
            console.print(f"  {name}: [green]✅ Configured[/green]")
        else:
            console.print(f"  {name}: [yellow]⚠️ Not set (optional for local models)[/
yellow]")

if __name__ == "__main__":
    console.print("[bold blue]🔍 Generative AI Environment Verification[/bold blue]
\n")
    console.print(f"Python: {sys.version}\n")
    check_imports()
    check_api_keys()
```

Run it:

```
pixi run python src/verify_setup.py
```

🛠️ Project 1: Environment Health Dashboard

Goal: Build a CLI tool that checks your entire AI development environment.

Extend `verify_setup.py` to also:

1. Check available GPU/CUDA status using `torch.cuda.is_available()`
2. Estimate available VRAM for local model selection
3. Test network connectivity to OpenAI and Anthropic APIs
4. Generate a `system_report.json` with all findings

```python
# src/health_dashboard.py
"""Complete environment health dashboard."""

import json
import os
import sys
import time
from datetime import datetime

def check_gpu():
    """Check GPU availability and VRAM."""
    try:
        import torch
        if torch.cuda.is_available():
            gpu_name = torch.cuda.get_device_name(0)
            vram_gb = torch.cuda.get_device_properties(0).total_mem / (1024**3)
            return {
                "available": True,
                "name": gpu_name,
                "vram_gb": round(vram_gb, 1),
                "cuda_version": torch.version.cuda,
            }
        elif hasattr(torch.backends, "mps") and torch.backends.mps.is_available():
            return {"available": True, "name": "Apple Silicon (MPS)", "vram_gb": "shar
ed"}
        return {"available": False, "name": "CPU only", "vram_gb": 0}
    except Exception as e:
        return {"available": False, "error": str(e)}

def check_api_connectivity():
    """Test connectivity to AI API providers."""
    import requests
    results = {}
    endpoints = {
        "OpenAI": "https://api.openai.com/v1/models",
        "Anthropic": "https://api.anthropic.com/v1/messages",
        "HuggingFace": "https://huggingface.co/api/models?limit=1",
    }
    for name, url in endpoints.items():
        try:
            start = time.time()
            resp = requests.get(url, timeout=5)
            latency_ms = round((time.time() - start) * 1000)
            results[name] = {
                "reachable": True,
                "status_code": resp.status_code,
                "latency_ms": latency_ms,
            }
        except Exception as e:
            results[name] = {"reachable": False, "error": str(e)}
    return results

def generate_report():
    """Generate a comprehensive system report."""
    report = {
        "timestamp": datetime.now().isoformat(),
        "python_version": sys.version,
        "platform": sys.platform,
        "gpu": check_gpu(),
        "api_connectivity": check_api_connectivity(),
    }
```

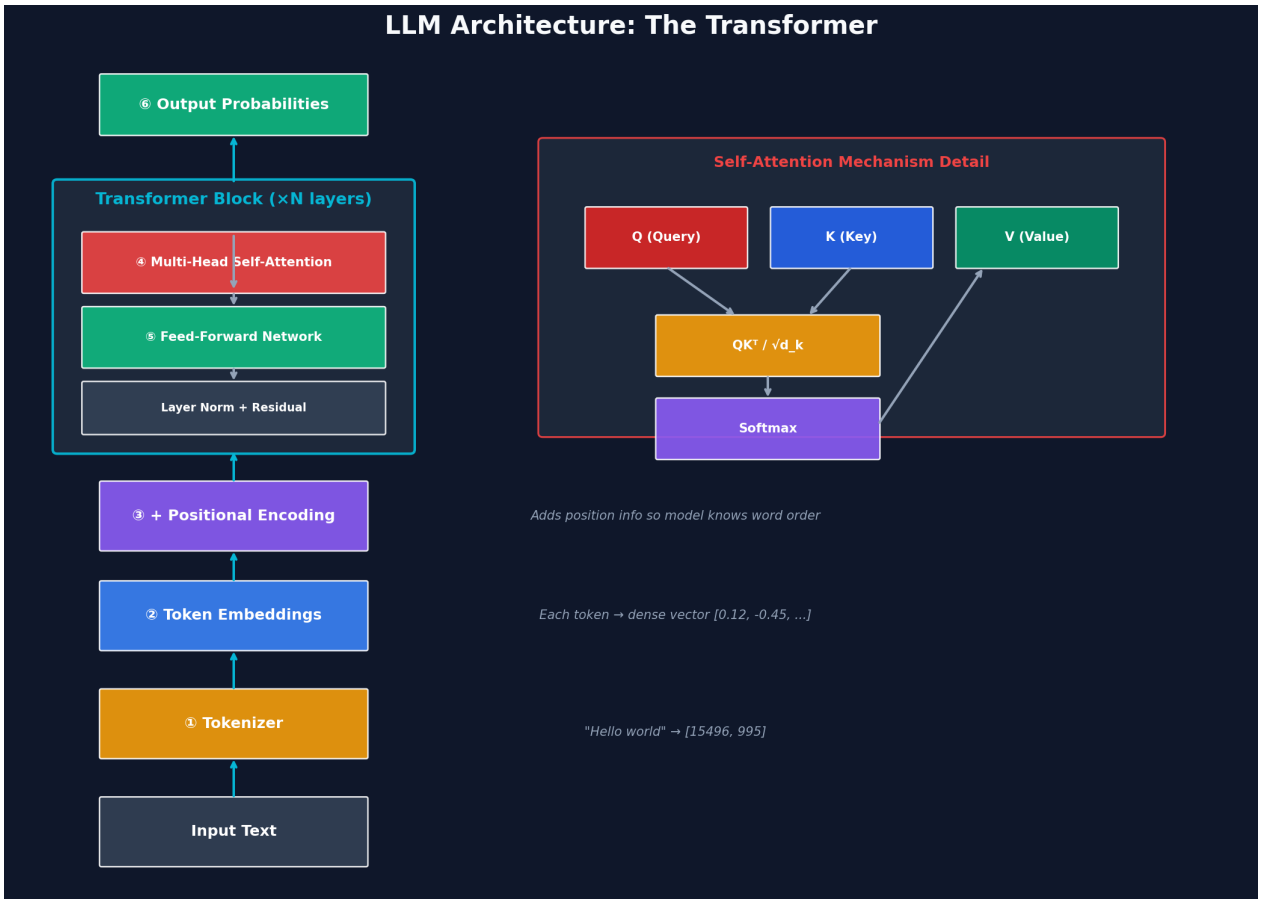
```
    # Check recommended model sizes based on available resources
    gpu_info = report["gpu"]
    if gpu_info.get("vram_gb", 0) and isinstance(gpu_info["vram_gb"], (int, float)):
        vram = gpu_info["vram_gb"]
        if vram >= 24:
            report["recommended_local_models"] = ["Llama-3-70B (4-bit)", "Mistral-
Large"]
        elif vram >= 8:
            report["recommended_local_models"] = ["Llama-3-8B", "Mistral-7B", "Phi-3-
mini"]
        elif vram >= 4:
            report["recommended_local_models"] = ["Phi-3-mini (4-bit)", "TinyL-
lama-1.1B"]
        else:
            report["recommended_local_models"] = ["CPU models only — use APIs in-
stead"]
    else:
        report["recommended_local_models"] = ["Use APIs or Apple MPS acceleration"]

    # Save report
    with open("system_report.json", "w") as f:
        json.dump(report, f, indent=2)

    return report

if __name__ == "__main__":
    from rich.console import Console
    from rich.json import JSON
    console = Console()
    console.print("[bold]🏥 AI Development Environment Health Report[/bold]\n")
    report = generate_report()
    console.print(JSON(json.dumps(report, indent=2)))
    console.print(f"\n[green]Report saved to system_report.json[/green]")
```

---

3. Understanding Large Language Models (LLMs)

3.1 The Transformer Architecture

Every modern LLM is built on the **Transformer** architecture, introduced in the landmark 2017 paper “Attention Is All You Need” by Vaswani et al. Understanding this architecture is essential for working effectively with LLMs.



The key components of a Transformer:

Tokenization — Turning Text into Numbers

LLMs don't read text — they process **tokens**. A tokenizer converts raw text into a sequence of integer IDs that the model understands.

```
# Exploring tokenization with the tiktoken library (used by OpenAI models)
# pixi add tiktoken

import tiktoken

# Load the tokenizer for GPT-4
encoder = tiktoken.encoding_for_model("gpt-4")

text = "Generative AI is transforming software development!"
tokens = encoder.encode(text)
print(f"Text: {text}")
print(f"Token IDs: {tokens}")
print(f"Number of tokens: {len(tokens)}")

# Decode individual tokens to see the breakdown
for token_id in tokens:
    decoded = encoder.decode([token_id])
    print(f"  {token_id:>6} → '{decoded}'")
```

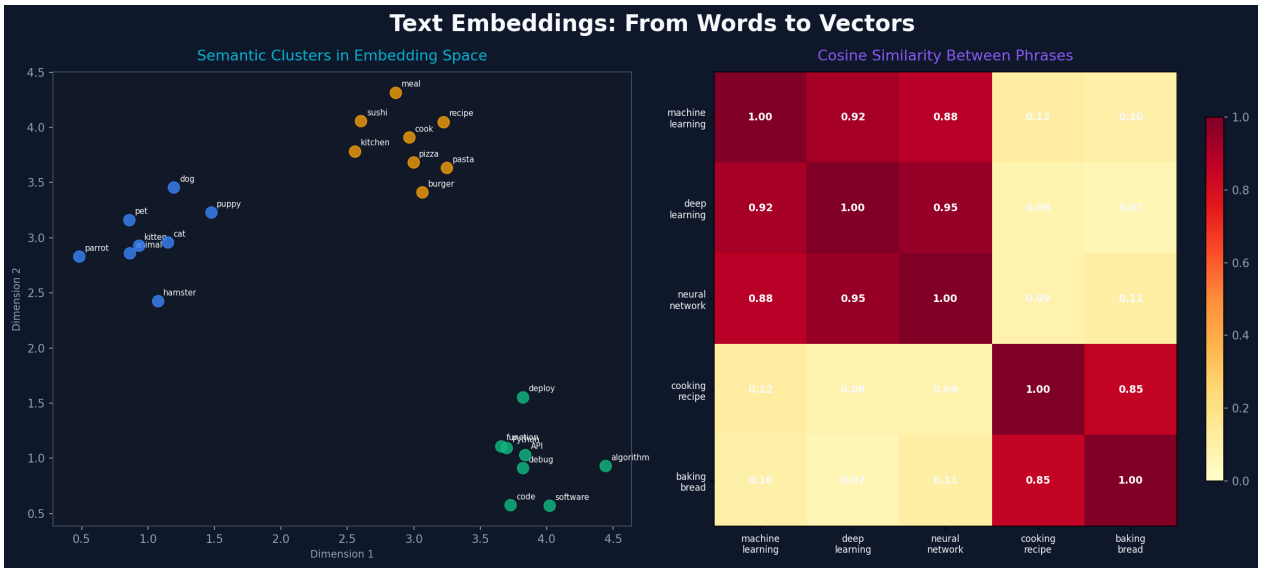
Output:

```
Text: Generative AI is transforming software development!
Token IDs: [5765, 1413, 15592, 374, 46890, 3241, 4500, 0]
Number of tokens: 8
   5765 → 'Gener'
   1413 → 'ative'
  15592 → ' AI'
    374 → ' is'
  46890 → ' transforming'
   3241 → ' software'
   4500 → ' development'
      0 → '!'
```

Key insight: Tokens aren't always whole words. Common words may be a single token, while rare words get split into subword pieces. This is critical for understanding context windows and API pricing.

Embeddings — Meaning as Vectors

After tokenization, each token ID is mapped to a high-dimensional **embedding vector** (typically 768–12,288 dimensions). These vectors encode semantic meaning — words with similar meanings have similar vectors.



```python
# Visualizing embeddings with sentence-transformers
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")

sentences = [
    "The cat sat on the mat",
    "A kitten rested on the rug",
    "Python is a programming language",
    "JavaScript is used for web development",
    "The weather is sunny today",
]

# Generate embeddings
embeddings = model.encode(sentences)
print(f"Embedding shape: {embeddings.shape}")  # (5, 384)

# Calculate cosine similarity between all pairs
from numpy.linalg import norm

def cosine_similarity(a, b):
    return np.dot(a, b) / (norm(a) * norm(b))

print("\nSemantic Similarity Matrix:")
print(f"{'':>40}", end="")
for i in range(len(sentences)):
    print(f"  [{i}]", end="")
print()

for i, s1 in enumerate(sentences):
    print(f"[{i}] {s1[:37]:>40}", end="")
    for j, s2 in enumerate(sentences):
        sim = cosine_similarity(embeddings[i], embeddings[j])
        print(f" {sim:.2f}", end="")
    print()
```

The Self-Attention Mechanism

The **self-attention** mechanism is the breakthrough that makes Transformers work. It lets each token "attend to" every other token in the sequence, computing relevance scores to determine which other words matter most for understanding the current word.

How it works:

1. Each token embedding is projected into three vectors: **Query (Q)**, **Key (K)**, and **Value (V)**
2. Attention scores are computed: `Attention(Q, K, V) = softmax(QK^T / √d_k) · V`
3. The softmax creates a probability distribution — a "heat map" of which tokens to focus on
4. **Multi-head attention** runs this process in parallel with different learned projections, capturing different relationship types (syntax, semantics, coreference, etc.)

```python
# Simplified self-attention implementation for educational purposes
import torch
import torch.nn.functional as F

def simple_self_attention(embeddings, d_k=64):
    """
    Simplified single-head self-attention.

    Args:
        embeddings: Tensor of shape (seq_len, d_model) — token embeddings
        d_k: Dimension of query/key projections
    """
    seq_len, d_model = embeddings.shape

    # Learnable projection matrices (normally these are nn.Linear layers)
    W_Q = torch.randn(d_model, d_k)
    W_K = torch.randn(d_model, d_k)
    W_V = torch.randn(d_model, d_k)

    # Project embeddings into Q, K, V spaces
    Q = embeddings @ W_Q   # (seq_len, d_k)
    K = embeddings @ W_K   # (seq_len, d_k)
    V = embeddings @ W_V   # (seq_len, d_k)

    # Compute attention scores: QK^T / sqrt(d_k)
    scores = (Q @ K.T) / (d_k ** 0.5)   # (seq_len, seq_len)

    # Apply softmax to get attention weights (probabilities)
    attention_weights = F.softmax(scores, dim=-1)   # (seq_len, seq_len)

    # Weighted sum of values
    output = attention_weights @ V   # (seq_len, d_k)

    return output, attention_weights

# Example: 4 tokens, each with 8-dimensional embeddings
torch.manual_seed(42)
token_embeddings = torch.randn(4, 8)
output, weights = simple_self_attention(token_embeddings, d_k=4)

print("Attention weights (each row sums to 1.0):")
print(weights.detach().numpy().round(3))
print(f"\nRow sums: {weights.sum(dim=-1).detach().numpy().round(3)}")
```

3.2 Types of LLMs

Understanding the different LLM families helps you choose the right model for each task:

Model	Provider	Key Strengths	Context Window	Best For
GPT-4o / GPT-5	OpenAI	Multimodal, fast, versatile	128K–1M tokens	General-purpose, vision tasks
Claude Opus / Sonnet	Anthropic	Long context, safety, analysis	200K tokens	Research, long documents, code
Gemini 2.5	Google	Multimodal, grounding	1M+ tokens	Multimodal, Google ecosystem
Llama 3	Meta	Open-source, customizable	128K tokens	Self-hosting, fine-tuning
Mistral / Mixtral	Mistral AI	Efficient, open-weight	32K–128K tokens	Cost-effective deployment
Phi-3 / Phi-4	Microsoft	Small but capable	4K–128K tokens	Edge devices, low resources

3.3 Key LLM Concepts You Must Know

Temperature controls randomness. Temperature=0 is deterministic (always picks the most likely token). Temperature=1.0 adds creativity. Higher values increase randomness.

Top-p (nucleus sampling) keeps only the smallest set of tokens whose cumulative probability exceeds p. Top-p=0.9 means the model samples from the top 90% probability mass.

Context Window is the maximum number of tokens the model can process at once (input + output). A 128K context window can handle ~300 pages of text.

System Prompts set the model’s persona and behavior rules. They are the most powerful tool for controlling output quality.

```python
# Demonstrating temperature effects
from openai import OpenAI
import os
from dotenv import load_dotenv

load_dotenv()
client = OpenAI()

prompt = "Write a one-sentence tagline for a coffee shop."

for temp in [0.0, 0.5, 1.0, 1.5]:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=temp,
        max_tokens=50,
    )
    print(f"Temperature {temp}: {response.choices[0].message.content}")
```

🛠️ Project 2: Token Economy Analyzer

Goal: Build a tool that analyzes the token cost of different prompts across models.

```python
# src/token_analyzer.py
"""Analyze token counts and estimated costs across LLM providers."""

import tiktoken
from rich.console import Console
from rich.table import Table

console = Console()

# Pricing per 1M tokens (approximate, as of early 2026)
PRICING = {
    "gpt-4o": {"input": 2.50, "output": 10.00},
    "gpt-4o-mini": {"input": 0.15, "output": 0.60},
    "claude-3.5-sonnet": {"input": 3.00, "output": 15.00},
    "claude-3-haiku": {"input": 0.25, "output": 1.25},
}

def count_tokens(text: str, model: str = "gpt-4") -> int:
    """Count tokens for a given text using tiktoken."""
    try:
        encoding = tiktoken.encoding_for_model(model)
    except KeyError:
        encoding = tiktoken.get_encoding("cl100k_base")
    return len(encoding.encode(text))

def analyze_prompt(prompt: str, expected_output_tokens: int = 500):
    """Analyze cost of a prompt across different models."""
    input_tokens = count_tokens(prompt)

    table = Table(title=f"Token Economy Analysis")
    table.add_column("Model", style="cyan")
    table.add_column("Input Tokens", justify="right")
    table.add_column("Est. Output Tokens", justify="right")
    table.add_column("Input Cost", justify="right", style="yellow")
    table.add_column("Output Cost", justify="right", style="yellow")
    table.add_column("Total Cost", justify="right", style="bold green")

    for model, prices in PRICING.items():
        input_cost = (input_tokens / 1_000_000) * prices["input"]
        output_cost = (expected_output_tokens / 1_000_000) * prices["output"]
        total = input_cost + output_cost
        table.add_row(
            model,
            str(input_tokens),
            str(expected_output_tokens),
            f"${input_cost:.6f}",
            f"${output_cost:.6f}",
            f"${total:.6f}",
        )

    console.print(f"\n[bold]Prompt preview:[/bold] {prompt[:100]}...")
    console.print(f"[bold]Character count:[/bold] {len(prompt)}")
    console.print(table)

if __name__ == "__main__":
    # Example: analyze a RAG prompt with context
    sample_prompt =
"""You are a helpful assistant that answers questions based on the
    provided context. Use only the information in the context to answer.

    Context:
    The transformer architecture was introduced in 2017 in the paper "Attention Is All
```

```
    You Need" by Vaswani et al. It replaced recurrent neural networks as the dominant
    architecture for natural language processing tasks. The key innovation was the
    self-attention mechanism, which allows the model to process all tokens in parallel

    rather than sequentially. This led to massive speedups in training and enabled the

    creation of much larger models. Modern LLMs like GPT-4, Claude, and Llama are all
    based on the transformer architecture.

    Question: What was the key innovation of the transformer architecture?"""

    analyze_prompt(sample_prompt, expected_output_tokens=200)
```

---

4. Working with LLM APIs

4.1 OpenAI API — Complete Guide

The OpenAI API is the most widely used LLM API. Let's explore it comprehensively.

Getting your API key:

1. Go to platform.openai.com (https://platform.openai.com)
2. Sign up or log in
3. Navigate to API Keys → Create new secret key
4. Copy the key to your `.env` file

Basic Chat Completion

```
# src/openai_basics.py
"""Complete guide to OpenAI API usage."""

from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()
client = OpenAI()  # Automatically reads OPENAI_API_KEY from environment

# --- Basic Chat Completion ---
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {
            "role": "system",
            "content": "You are a concise Python tutor. Explain concepts clearly with
examples.",
        },
        {
            "role": "user",
            "content": "What are Python decorators?",
        },
    ],
    temperature=0.7,
    max_tokens=500,
)

print(response.choices[0].message.content)
print(f"\nTokens used — Input: {response.usage.prompt_tokens}, "
      f"Output: {response.usage.completion_tokens}, "
      f"Total: {response.usage.total_tokens}")
```

Streaming Responses

Streaming provides a much better user experience — text appears word by word instead of waiting for the full response:

```python
# --- Streaming Chat Completion ---
def stream_chat(messages: list[dict], model: str = "gpt-4o-mini"):
    """Stream a chat completion and print tokens as they arrive."""
    stream = client.chat.completions.create(
        model=model,
        messages=messages,
        stream=True,
    )

    full_response = ""
    for chunk in stream:
        if chunk.choices[0].delta.content is not None:
            token = chunk.choices[0].delta.content
            print(token, end="", flush=True)
            full_response += token

    print()  # Newline after streaming completes
    return full_response

# Usage
result = stream_chat([
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Write a haiku about machine learning."},
])
```

Multi-Turn Conversations

```python
# --- Multi-turn Conversation Manager ---
class ChatSession:
    """Manages a multi-turn conversation with an LLM."""

    def __init__(self, model: str = "gpt-4o-mini", system_prompt: str = ""):
        self.model = model
        self.messages = []
        if system_prompt:
            self.messages.append({"role": "system", "content": system_prompt})

    def chat(self, user_message: str) -> str:
        """Send a message and get a response, maintaining conversation history."""
        self.messages.append({"role": "user", "content": user_message})

        response = client.chat.completions.create(
            model=self.model,
            messages=self.messages,
            temperature=0.7,
        )

        assistant_message = response.choices[0].message.content
        self.messages.append({"role": "assistant", "content": assistant_message})
        return assistant_message

    def get_history(self) -> list[dict]:
        """Return the full conversation history."""
        return self.messages.copy()

    def clear(self):
        """Clear conversation history (keep system prompt)."""
        self.messages = [m for m in self.messages if m["role"] == "system"]

# Usage
session = ChatSession(
    system_prompt="You are an expert Python developer. Be concise."
)
print(session.chat("What's the difference between a list and a tuple?"))
print("---")
print(session.chat("Show me a practical example of when to use each."))
print("---")
print(session.chat("Now explain which is more memory efficient and why."))
```

Structured Output with JSON Mode

```
# --- Structured Output ---
import json

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {
            "role": "system",
            "content": "You analyze text sentiment. Return JSON with keys: "
                       "sentiment (positive/negative/neutral), confidence (0-1), "
                       "key_phrases (list of strings).",
        },
        {
            "role": "user",
            "content": "I absolutely love this new AI framework! It makes development
"
                       "so much easier and the documentation is fantastic.",
        },
    ],
    response_format={"type": "json_object"},
    temperature=0,
)

result = json.loads(response.choices[0].message.content)
print(json.dumps(result, indent=2))
# Output:
# {
#   "sentiment": "positive",
#   "confidence": 0.95,
#   "key_phrases": ["absolutely love", "so much easier", "documentation is fantastic"]
# }
```

4.2 Anthropic Claude API

Anthropic’s Claude excels at long-context analysis, careful reasoning, and nuanced tasks.

```python
# src/anthropic_basics.py
"""Complete guide to Anthropic Claude API usage."""

import anthropic
from dotenv import load_dotenv

load_dotenv()
client = anthropic.Anthropic()  # Reads ANTHROPIC_API_KEY from environment

# --- Basic Message ---
message = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    system="You are a world-class technical writer. Explain complex topics simply.",
    messages=[
        {
            "role": "user",
            "content": "Explain how RAG works in 3 bullet points.",
        }
    ],
)

print(message.content[0].text)
print(f"\nTokens — Input: {message.usage.input_tokens}, "
      f"Output: {message.usage.output_tokens}")
```

Claude Streaming

```python
# --- Streaming with Claude ---
def stream_claude(prompt: str, system: str = "", model: str = "claude-son-
net-4-20250514"):
    """Stream a Claude response."""
    with client.messages.stream(
        model=model,
        max_tokens=1024,
        system=system,
        messages=[{"role": "user", "content": prompt}],
    ) as stream:
        full_text = ""
        for text in stream.text_stream:
            print(text, end="", flush=True)
            full_text += text
        print()
        return full_text

stream_claude(
    "Write a Python function that implements binary search with detailed comments.",
    system="You are an expert Python developer. Write clean, well-documented code."
)
```

4.3 Building a Multi-Provider Chat Interface

Let’s build a unified interface that works with any provider:

```python
# src/multi_provider_chat.py
"""Unified multi-provider chat interface."""

from abc import ABC, abstractmethod
from dataclasses import dataclass
from dotenv import load_dotenv

load_dotenv()

@dataclass
class ChatResponse:
    """Standardized response from any LLM provider."""
    content: str
    model: str
    input_tokens: int
    output_tokens: int
    provider: str

class LLMProvider(ABC):
    """Abstract base class for LLM providers."""

    @abstractmethod
    def chat(self, messages: list[dict], **kwargs) -> ChatResponse:
        pass

class OpenAIProvider(LLMProvider):
    def __init__(self, model: str = "gpt-4o-mini"):
        from openai import OpenAI
        self.client = OpenAI()
        self.model = model

    def chat(self, messages: list[dict], **kwargs) -> ChatResponse:
        response = self.client.chat.completions.create(
            model=self.model,
            messages=messages,
            temperature=kwargs.get("temperature", 0.7),
            max_tokens=kwargs.get("max_tokens", 1024),
        )
        return ChatResponse(
            content=response.choices[0].message.content,
            model=self.model,
            input_tokens=response.usage.prompt_tokens,
            output_tokens=response.usage.completion_tokens,
            provider="OpenAI",
        )

class AnthropicProvider(LLMProvider):
    def __init__(self, model: str = "claude-sonnet-4-20250514"):
        import anthropic
        self.client = anthropic.Anthropic()
        self.model = model

    def chat(self, messages: list[dict], **kwargs) -> ChatResponse:
        # Extract system message if present
        system = ""
        chat_messages = []
        for msg in messages:
            if msg["role"] == "system":
                system = msg["content"]
            else:
                chat_messages.append(msg)
```

```
        response = self.client.messages.create(
            model=self.model,
            max_tokens=kwargs.get("max_tokens", 1024),
            system=system,
            messages=chat_messages,
        )
        return ChatResponse(
            content=response.content[0].text,
            model=self.model,
            input_tokens=response.usage.input_tokens,
            output_tokens=response.usage.output_tokens,
            provider="Anthropic",
        )

class MultiProviderChat:
    """Unified chat interface supporting multiple LLM providers."""

    PROVIDERS = {
        "openai": OpenAIProvider,
        "anthropic": AnthropicProvider,
    }

    def __init__(self, provider: str = "openai", model: str = None):
        if provider not in self.PROVIDERS:
            raise ValueError(f"Unknown provider: {provider}. Options: {list(self.PRO-
VIDERS)}")
        kwargs = {"model": model} if model else {}
        self.provider = self.PROVIDERS[provider](**kwargs)
        self.history: list[dict] = []

    def set_system_prompt(self, prompt: str):
        self.history = [{"role": "system", "content": prompt}]

    def send(self, message: str, **kwargs) -> ChatResponse:
        self.history.append({"role": "user", "content": message})
        response = self.provider.chat(self.history, **kwargs)
        self.history.append({"role": "assistant", "content": response.content})
        return response

if __name__ == "__main__":
    from rich.console import Console
    console = Console()

    # Use with OpenAI
    chat = MultiProviderChat(provider="openai", model="gpt-4o-mini")
    chat.set_system_prompt("You are a helpful AI assistant. Be concise.")

    response = chat.send("What is the capital of France?")
    console.print(f"[cyan][{response.provider}/{response.model}][/cyan] {response.con-
tent}")
    console.print(f"[dim]Tokens: {response.input_tokens} in /
{response.output_tokens} out[/dim]")
```

🛠️ Project 3: Interactive CLI Chatbot

Goal: Build a fully interactive terminal chatbot that supports provider switching, conversation saving, and token tracking.

```python
# src/chatbot.py
"""Interactive multi-provider CLI chatbot with session management."""

import json
import os
from datetime import datetime
from rich.console import Console
from rich.prompt import Prompt
from rich.panel import Panel
from rich.markdown import Markdown

# Assumes MultiProviderChat is importable from the module above
# from multi_provider_chat import MultiProviderChat, ChatResponse

console = Console()

class InteractiveChatbot:
    """Full-featured interactive chatbot with session management."""

    def __init__(self):
        self.chat = None
        self.total_input_tokens = 0
        self.total_output_tokens = 0
        self.session_start = datetime.now()
        self.conversation_log = []

    def initialize(self):
        """Set up the chatbot with user preferences."""
        console.print(Panel(
            "[bold blue]🤖 AI Chatbot — Multi-Provider Edition[/bold blue]\n"
            "Choose your AI provider and start chatting!",
            border_style="blue"
        ))

        provider = Prompt.ask(
            "Select provider",
            choices=["openai", "anthropic"],
            default="openai"
        )

        model_defaults = {"openai": "gpt-4o-mini", "anthropic": "claude-son-
net-4-20250514"}
        model = Prompt.ask("Model", default=model_defaults[provider])

        self.chat = MultiProviderChat(provider=provider, model=model)
        self.chat.set_system_prompt(
            "You are a helpful, knowledgeable AI assistant. "
            "Format your responses in markdown when appropriate."
        )
        console.print(f"\n[green]✅ Connected to {provider}/{model}[/green]")
        console.print("[dim]Commands: /save, /clear, /stats, /quit[/dim]\n")

    def handle_command(self, command: str) -> bool:
        """Handle slash commands. Returns True if should continue, False to quit."""
        if command == "/quit":
            return False
        elif command == "/clear":
            self.chat.history = [m for m in self.chat.history if m["role"] ==
"system"]
            console.print("[yellow]Conversation cleared.[/yellow]")
        elif command == "/stats":
            elapsed = (datetime.now() - self.session_start).seconds
```

```
            console.print(Panel(
                f"Total input tokens: {self.total_input_tokens}\n"
                f"Total output tokens: {self.total_output_tokens}\n"
                f"Messages exchanged: {len(self.conversation_log)}\n"
                f"Session duration: {elapsed // 60}m {elapsed % 60}s",
                title="Session Statistics", border_style="cyan"
            ))
        elif command == "/save":
            filename = f"chat_{datetime.now().strftime('%Y%m%d_%H%M%S')}.json"
            with open(filename, "w") as f:
                json.dump(self.conversation_log, f, indent=2)
            console.print(f"[green]Conversation saved to {filename}[/green]")
        else:
            console.print(f"[red]Unknown command: {command}[/red]")
        return True

    def run(self):
        """Main chat loop."""
        self.initialize()

        while True:
            try:
                user_input = Prompt.ask("[bold green]You[/bold green]")
            except (KeyboardInterrupt, EOFError):
                break

            if not user_input.strip():
                continue

            if user_input.startswith("/"):
                if not self.handle_command(user_input.strip()):
                    break
                continue

            try:
                response = self.chat.send(user_input)
                self.total_input_tokens += response.input_tokens
                self.total_output_tokens += response.output_tokens
                self.conversation_log.append({
                    "role": "user", "content": user_input,
                    "timestamp": datetime.now().isoformat()
                })
                self.conversation_log.append({
                    "role": "assistant", "content": response.content,
                    "model": response.model, "tokens": response.output_tokens,
                    "timestamp": datetime.now().isoformat()
                })

                console.print(f"\n[bold blue]🤖 {response.provider}[/bold blue]")
                console.print(Markdown(response.content))
                console.print(f"[dim]({response.input_tokens}+
{response.output_tokens} tokens)[/dim]\n")

            except Exception as e:
                console.print(f"[bold red]Error: {e}[/bold red]")

        console.print("\n[blue]Thanks for chatting! Goodbye. 👋[/blue]")

if __name__ == "__main__":
    bot = InteractiveChatbot()
    bot.run()
```

5. Local LLMs with HuggingFace

Running LLMs locally gives you **full control**, **data privacy**, **zero API costs**, and the ability to **fine-tune** models on your own data.

5.1 HuggingFace Transformers — Getting Started

The `transformers` library by HuggingFace provides a unified Python API for over a million models.

```python
# src/local_llm.py
"""Working with local LLMs using HuggingFace Transformers."""

from transformers import pipeline

# The `pipeline` function is the simplest way to use local models.
# On first run, it downloads the model (may take several minutes).

# --- Text Generation with a Small Model ---
generator = pipeline(
    "text-generation",
    model="microsoft/Phi-3-mini-4k-instruct",
    device_map="auto",     # Automatically use GPU if available
    torch_dtype="auto",    # Use optimal precision
)

messages = [
    {"role": "system", "content": "You are a helpful Python tutor."},
    {"role": "user", "content": "Explain list comprehensions with 3 examples."},
]

output = generator(
    messages,
    max_new_tokens=512,
    temperature=0.7,
    do_sample=True,
)
print(output[0]["generated_text"][-1]["content"])
```

5.2 Choosing the Right Local Model

Local model selection depends on your hardware:

Your Hardware	Recommended Models	VRAM Needed
No GPU (CPU only)	TinyLlama-1.1B, Phi-3-mini (GGUF/4-bit)	2-4 GB RAM
4-6 GB VRAM	Phi-3-mini-4k, Gemma-2B	4-6 GB
8-12 GB VRAM	Llama-3-8B, Mistral-7B, Phi-3-medium	8-12 GB
16-24 GB VRAM	Llama-3-8B (full precision), CodeLlama-13B	16-24 GB
48+ GB VRAM	Llama-3-70B (4-bit), Mixtral-8x7B	40+ GB
Apple Silicon (M1/M2/M3)	Most 7-8B models via MPS backend	Shared memory

5.3 Quantization — Running Large Models on Limited Hardware

Quantization reduces model precision from 32-bit or 16-bit floating point to 8-bit or 4-bit integers. This dramatically reduces memory usage with minimal quality loss.

```
# --- Loading a Quantized Model with BitsAndBytes ---
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
import torch

# 4-bit quantization config — reduces memory by ~4x
quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4",              # Normalized float 4-bit
    bnb_4bit_use_double_quant=True,          # Double quantization for extra savings
)

model_name = "meta-llama/Meta-Llama-3-8B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=quantization_config,
    device_map="auto",
)

# Generate text
prompt = "Explain quantum computing in simple terms:"
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
outputs = model.generate(**inputs, max_new_tokens=200, temperature=0.7,
do_sample=True)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

5.4 Using Sentence Transformers for Embeddings

Sentence Transformers create high-quality embeddings for semantic search — a critical component for RAG.

```
# --- Generating and Using Embeddings Locally ---
from sentence_transformers import SentenceTransformer
import numpy as np

# Load a fast, high-quality embedding model
embed_model = SentenceTransformer("all-MiniLM-L6-v2")  # 80MB, very fast
# For higher quality: "BAAI/bge-large-en-v1.5" (1.3GB)

# Encode documents
documents = [
    "Python was created by Guido van Rossum in 1991.",
    "JavaScript is the most popular language for web development.",
    "Machine learning is a subset of artificial intelligence.",
    "The Eiffel Tower is located in Paris, France.",
    "Neural networks are inspired by biological brain structures.",
]

doc_embeddings = embed_model.encode(documents, show_progress_bar=True)
print(f"Embedding shape: {doc_embeddings.shape}")  # (5, 384)

# Semantic search function
def semantic_search(query: str, docs: list[str], embeddings: np.ndarray, top_k: int =
3):
    """Find the most semantically similar documents to a query."""
    query_embedding = embed_model.encode([query])
    # Cosine similarity
    similarities = np.dot(embeddings, query_embedding.T).flatten()
    top_indices = np.argsort(similarities)[::-1][:top_k]
    return [(docs[i], float(similarities[i])) for i in top_indices]

# Test it
results = semantic_search("What is deep learning?", documents, doc_embeddings)
for doc, score in results:
    print(f"  [{score:.3f}] {doc}")
```

🛠️ Project 4: Local Model Benchmarking Tool

Goal: Build a benchmarking tool that compares local models on speed, quality, and resource usage.

```python
# src/benchmark_local.py
"""Benchmark local LLMs on speed, memory, and quality."""

import time
import psutil
import torch
from transformers import pipeline
from rich.console import Console
from rich.table import Table

console = Console()

BENCHMARK_PROMPTS = [
    {
        "name": "Code Generation",
        "messages": [
            {"role": "user", "content": "Write a Python function to check if a string
is a palindrome."}
        ],
    },
    {
        "name": "Reasoning",
        "messages": [
            {"role": "user", "content": "If a train travels 120 miles in 2 hours, and
then 180 miles in 3 hours, what is its average speed for the entire trip?"}
        ],
    },
    {
        "name": "Summarization",
        "messages": [
            {"role": "user", "content": "Summarize the concept of blockchain in
exactly 2 sentences."}
        ],
    },
]

def benchmark_model(model_name: str, max_new_tokens: int = 200) -> dict:
    """Benchmark a single model across all test prompts."""
    console.print(f"\n[bold cyan]Loading {model_name}...[/bold cyan]")

    mem_before = psutil.Process().memory_info().rss / (1024**3)
    load_start = time.time()

    gen = pipeline(
        "text-generation",
        model=model_name,
        device_map="auto",
        torch_dtype="auto",
    )

    load_time = time.time() - load_start
    mem_after = psutil.Process().memory_info().rss / (1024**3)

    results = {
        "model": model_name,
        "load_time_s": round(load_time, 1),
        "memory_gb": round(mem_after - mem_before, 2),
        "benchmarks": [],
    }

    for prompt in BENCHMARK_PROMPTS:
        start = time.time()
```

```
        output = gen(prompt["messages"], max_new_tokens=max_new_tokens, do_sample=Fals
e)
        elapsed = time.time() - start
        response_text = output[0]["generated_text"][-1]["content"]
        tokens_generated = len(gen.tokenizer.encode(response_text))

        results["benchmarks"].append({
            "task": prompt["name"],
            "time_s": round(elapsed, 2),
            "tokens": tokens_generated,
            "tokens_per_sec": round(tokens_generated / elapsed, 1),
            "response_preview": response_text[:100],
        })

    # Cleanup
    del gen
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

    return results

def run_benchmarks(models: list[str]):
    """Run benchmarks across multiple models and display results."""
    all_results = []
    for model in models:
        try:
            result = benchmark_model(model)
            all_results.append(result)
        except Exception as e:
            console.print(f"[red]Failed to benchmark {model}: {e}[/red]")

    # Display comparison table
    table = Table(title="Model Benchmark Comparison")
    table.add_column("Model", style="cyan")
    table.add_column("Load Time", justify="right")
    table.add_column("Memory", justify="right")
    table.add_column("Avg Tokens/sec", justify="right", style="green")

    for result in all_results:
        avg_tps = sum(b["tokens_per_sec"] for b in result["benchmarks"]) /
len(result["benchmarks"])
        table.add_row(
            result["model"].split("/")[-1],
            f"{result['load_time_s']}s",
            f"{result['memory_gb']} GB",
            f"{avg_tps:.1f}",
        )

    console.print(table)

if __name__ == "__main__":
    # Benchmark small models that work on most hardware
    models_to_test = [
        "microsoft/Phi-3-mini-4k-instruct",
        # Add more models as your hardware allows:
        # "mistralai/Mistral-7B-Instruct-v0.3",
        # "meta-llama/Meta-Llama-3-8B-Instruct",
    ]
    run_benchmarks(models_to_test)
```

6. Retrieval-Augmented Generation (RAG)

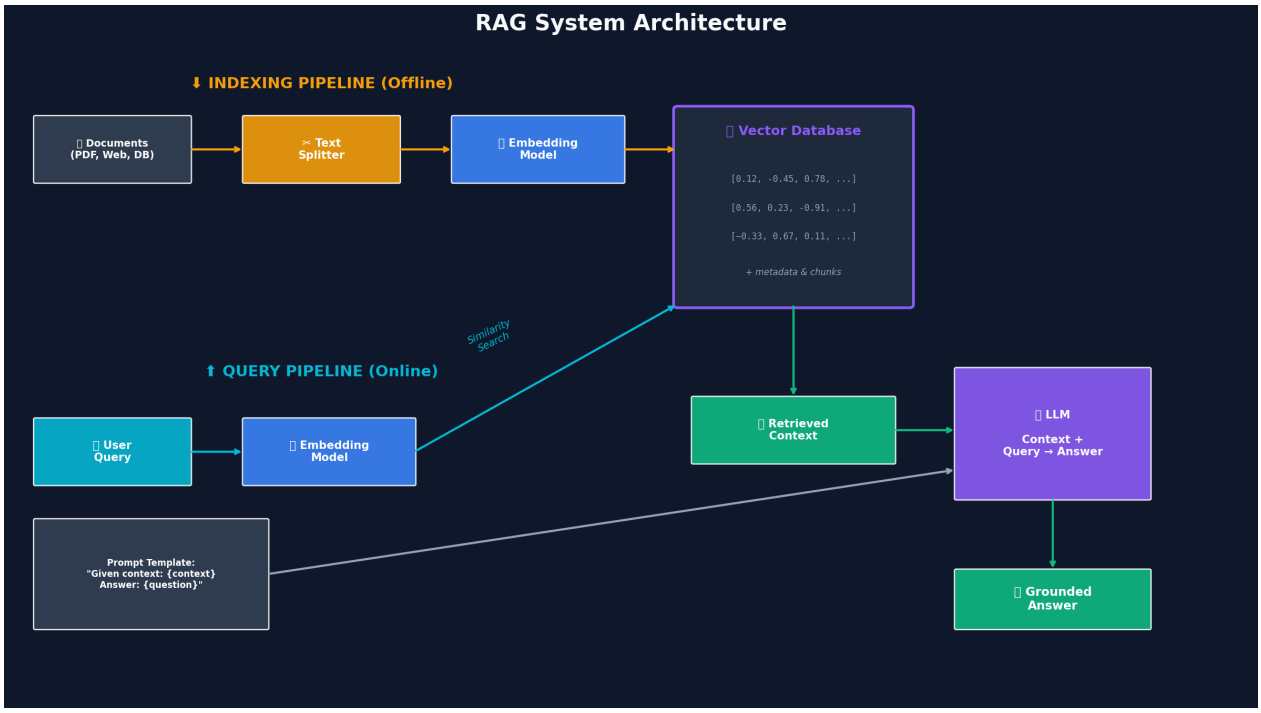
RAG is the technique that transforms LLMs from general knowledge systems into **domain-specific experts** that can answer questions about your data. This section covers RAG from basic concepts to production-ready implementations.

6.1 Why RAG? The Problem It Solves

LLMs have two fundamental limitations:

1. **Knowledge cutoff**: They only know what was in their training data. They can't answer about recent events or your private documents.
2. **Hallucination**: When they don't know something, they may confidently generate plausible-sounding but incorrect information.

RAG solves both problems by **retrieving relevant information** from a knowledge base and **injecting it into the prompt** before the LLM generates an answer.



6.2 The RAG Pipeline — Step by Step

A RAG system has two phases:

Phase 1: Indexing (Offline — done once)
1. **Load** documents (PDFs, web pages, databases, etc.)
2. **Split** documents into smaller, overlapping chunks
3. **Embed** each chunk using an embedding model → vectors
4. **Store** vectors in a vector database

Phase 2: Query (Online — happens per question)
1. **Embed** the user's question → query vector
2. **Search** the vector database for the most similar document chunks
3. **Augment** the prompt with retrieved chunks as context
4. **Generate** an answer using the LLM, grounded in the retrieved context

6.3 Building RAG with LangChain — Complete Implementation

```python
# src/rag_langchain.py
"""Complete RAG implementation with LangChain."""

import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.document_loaders import (
    PyPDFLoader,
    TextLoader,
    WebBaseLoader,
    DirectoryLoader,
)
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

load_dotenv()

# ── Step 1: Load Documents ──
def load_documents(source_path: str) -> list:
    """Load documents from various sources."""
    if source_path.startswith("http"):
        loader = WebBaseLoader(source_path)
    elif source_path.endswith(".pdf"):
        loader = PyPDFLoader(source_path)
    elif source_path.endswith(".txt") or source_path.endswith(".md"):
        loader = TextLoader(source_path)
    elif os.path.isdir(source_path):
        loader = DirectoryLoader(source_path, glob="**/*.txt")
    else:
        raise ValueError(f"Unsupported source: {source_path}")

    documents = loader.load()
    print(f"Loaded {len(documents)} document(s) from {source_path}")
    return documents

# ── Step 2: Split Documents into Chunks ──
def split_documents(documents: list, chunk_size: int = 1000, chunk_overlap: int = 200)
-> list:
    """Split documents into overlapping chunks for embedding."""
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        length_function=len,
        separators=["\n\n", "\n", ". ", " ", ""],  # Try to split at natural boundar-
ies
    )
    chunks = text_splitter.split_documents(documents)
    print(f"Split into {len(chunks)} chunks (size={chunk_size}, over-
lap={chunk_overlap})")
    return chunks

# ── Step 3: Create Vector Store ──
def create_vector_store(chunks: list, persist_directory: str = "./chroma_db") ->
Chroma:
    """Embed chunks and store in ChromaDB."""
    embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
    vectorstore = Chroma.from_documents(
        documents=chunks,
        embedding=embeddings,
```

```
        persist_directory=persist_directory,
    )
    print(f"Vector store created with {len(chunks)} vectors at {persist_directory}")
    return vectorstore

# ── Step 4: Build the RAG Chain ──
def build_rag_chain(vectorstore: Chroma, model: str = "gpt-4o-mini"):
    """Build the complete RAG chain using LCEL."""

    # Create a retriever that fetches the top 4 most relevant chunks
    retriever = vectorstore.as_retriever(
        search_type="similarity",
        search_kwargs={"k": 4},
    )

    # Define the RAG prompt template
    template = """You are a helpful assistant that answers questions based on the
provided context.
Use ONLY the information in the context below to answer the question. If the context
doesn't
contain enough information to answer, say "I don't have enough information to answer
that
based on the provided documents."

Context:
{context}

Question: {question}

Answer:"""

    prompt = ChatPromptTemplate.from_template(template)

    # Initialize the LLM
    llm = ChatOpenAI(model=model, temperature=0)

    # Helper to format retrieved documents
    def format_docs(docs):
        return "\n\n---\n\n".join(
            f"[Source: {doc.metadata.get('source', 'unknown')}]\n{doc.page_content}"
            for doc in docs
        )

    # Build the chain using LangChain Expression Language (LCEL)
    rag_chain = (
        {"context": retriever | format_docs, "question": RunnablePassthrough()}
        | prompt
        | llm
        | StrOutputParser()
    )

    return rag_chain, retriever

# ── Complete RAG Application ──
class RAGApplication:
    """Complete RAG application with indexing and querying."""

    def __init__(self, persist_dir: str = "./chroma_db"):
        self.persist_dir = persist_dir
        self.vectorstore = None
        self.chain = None
        self.retriever = None
```

```python
    def index(self, sources: list[str], chunk_size: int = 1000):
        """Index documents from multiple sources."""
        all_chunks = []
        for source in sources:
            docs = load_documents(source)
            chunks = split_documents(docs, chunk_size=chunk_size)
            all_chunks.extend(chunks)

        self.vectorstore = create_vector_store(all_chunks, self.persist_dir)
        self.chain, self.retriever = build_rag_chain(self.vectorstore)
        print(f"\n✅ Indexed {len(all_chunks)} chunks from {len(sources)} source(s)")

    def load_existing(self):
        """Load an existing vector store."""
        embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
        self.vectorstore = Chroma(
            persist_directory=self.persist_dir,
            embedding_function=embeddings,
        )
        self.chain, self.retriever = build_rag_chain(self.vectorstore)

    def query(self, question: str) -> dict:
        """Query the RAG system and return answer with sources."""
        if not self.chain:
            raise RuntimeError("No documents indexed. Call index() first.")

        # Get the answer
        answer = self.chain.invoke(question)

        # Get the source documents for transparency
        source_docs = self.retriever.invoke(question)
        sources = [
            {
                "content": doc.page_content[:200] + "...",
                "source": doc.metadata.get("source", "unknown"),
            }
            for doc in source_docs
        ]

        return {"answer": answer, "sources": sources}

# ── Usage Example ──
if __name__ == "__main__":
    from rich.console import Console
    from rich.panel import Panel
    console = Console()

    # Create and populate the RAG system
    rag = RAGApplication(persist_dir="./my_knowledge_base")

    # Index some documents (replace with your own sources)
    rag.index([
        "data/sample_document.txt",     # Local text files
        # "data/research_paper.pdf",    # PDF files
        # "https://example.com/article", # Web pages
    ])

    # Query the system
    result = rag.query("What are the main topics covered in the documents?")

    console.print(Panel(result["answer"], title="Answer", border_style="green"))
    console.print("\n[bold]Sources used:[/bold]")
```

```
    for i, source in enumerate(result["sources"], 1):
        console.print(f"  {i}. [{source['source']}] {source['content'][:80]}...")
```

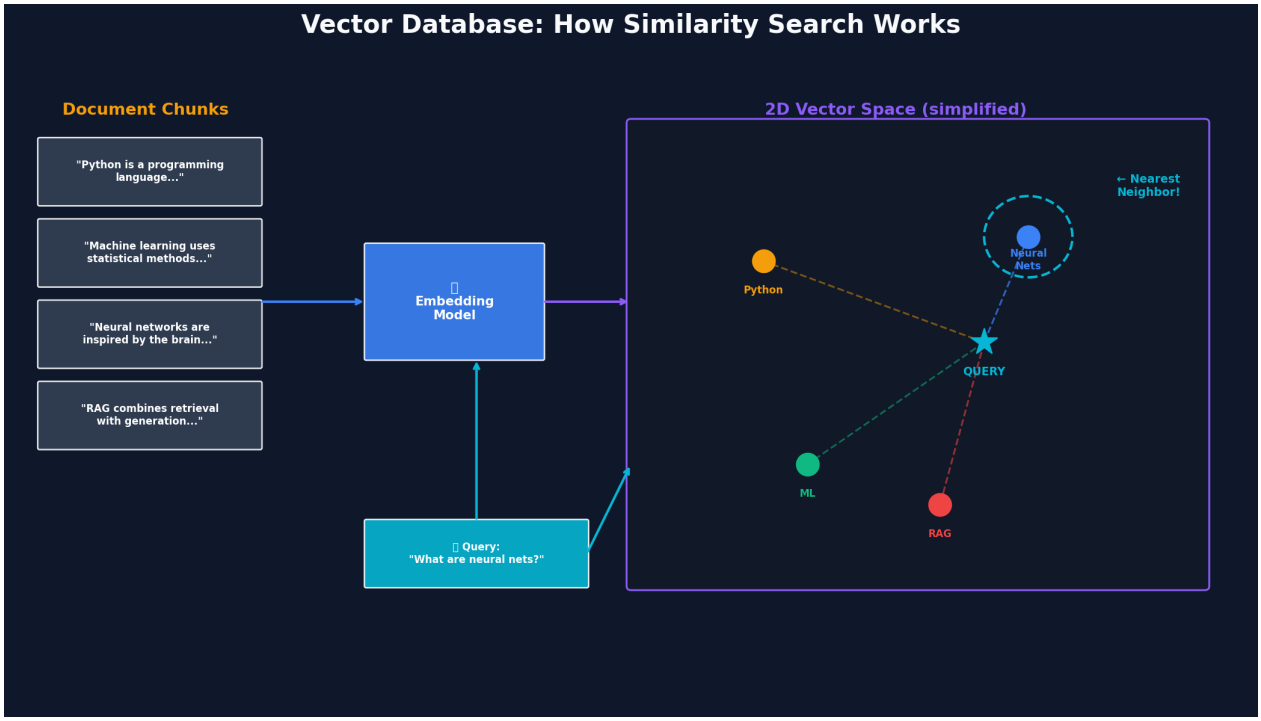
6.4 Building RAG with LlamaIndex

LlamaIndex provides a more data-focused approach to RAG:

```python
# src/rag_llamaindex.py
"""RAG implementation with LlamaIndex."""

import os
from dotenv import load_dotenv
from llama_index.core import (
    VectorStoreIndex,
    SimpleDirectoryReader,
    Settings,
    StorageContext,
    load_index_from_storage,
)
from llama_index.llms.openai import OpenAI
from llama_index.embeddings.openai import OpenAIEmbedding

load_dotenv()

# Configure global settings
Settings.llm = OpenAI(model="gpt-4o-mini", temperature=0)
Settings.embed_model = OpenAIEmbedding(model="text-embedding-3-small")
Settings.chunk_size = 1024
Settings.chunk_overlap = 200

class LlamaIndexRAG:
    """RAG application using LlamaIndex."""

    def __init__(self, storage_dir: str = "./llamaindex_storage"):
        self.storage_dir = storage_dir
        self.index = None

    def index_documents(self, data_dir: str):
        """Load and index documents from a directory."""
        # Load all documents from the directory
        documents = SimpleDirectoryReader(data_dir).load_data()
        print(f"Loaded {len(documents)} documents from {data_dir}")

        # Create vector index (handles chunking, embedding, storage)
        self.index = VectorStoreIndex.from_documents(
            documents,
            show_progress=True,
        )

        # Persist for later use
        self.index.storage_context.persist(persist_dir=self.storage_dir)
        print(f"Index persisted to {self.storage_dir}")

    def load_index(self):
        """Load a previously persisted index."""
        storage_context = StorageContext.from_defaults(persist_dir=self.storage_dir)
        self.index = load_index_from_storage(storage_context)

    def query(self, question: str, top_k: int = 4) -> dict:
        """Query the index."""
        query_engine = self.index.as_query_engine(
            similarity_top_k=top_k,
            response_mode="compact",     # Compact = synthesize from all retrieved
chunks
        )

        response = query_engine.query(question)

        sources = []
```

```
        for node in response.source_nodes:
            sources.append({
                "text": node.text[:200] + "...",
                "score": round(node.score, 3) if node.score else None,
                "file": node.metadata.get("file_name", "unknown"),
            })

        return {
            "answer": str(response),
            "sources": sources,
        }

if __name__ == "__main__":
    rag = LlamaIndexRAG()
    rag.index_documents("./data/")

    result = rag.query("Summarize the key points of the documents.")
    print(f"\nAnswer: {result['answer']}")
    print(f"\nSources: {len(result['sources'])} chunks used")
```

6.5 Vector Databases Deep Dive



Understanding vector databases is essential for building effective RAG systems:

```python
# src/vector_db_examples.py
"""Hands-on exploration of vector database operations."""

import chromadb
from sentence_transformers import SentenceTransformer

# Initialize ChromaDB (local, file-based)
client = chromadb.PersistentClient(path="./chroma_exploration")

# Create (or get) a collection
collection = client.get_or_create_collection(
    name="ai_concepts",
    metadata={"description": "AI and ML concepts for learning"},
)

# Prepare the embedding model
embed_model = SentenceTransformer("all-MiniLM-L6-v2")

# --- Adding Documents ---
documents = [
    "Supervised learning uses labeled data to train models for classification and re-
gression.",
    "Unsupervised learning finds patterns in data without labels, like clustering.",
    "Reinforcement learning trains agents through trial and error with rewards.",
    "Transfer learning reuses a pre-trained model on a new but related task.",
    "Transformers use self-attention to process sequences in parallel.",
    "Convolutional neural networks are designed for processing grid-like data such as
images.",
    "Recurrent neural networks process sequential data using internal memory.",
    "Generative Adversarial Networks pit two networks against each other to generate
data.",
    "Gradient descent is an optimization algorithm that minimizes loss functions.",
    "Backpropagation computes gradients efficiently through the chain rule.",
]

# Generate embeddings and add to ChromaDB
embeddings = embed_model.encode(documents).tolist()

collection.add(
    ids=[f"doc_{i}" for i in range(len(documents))],
    documents=documents,
    embeddings=embeddings,
    metadatas=[{"topic": "ml_fundamentals", "index": i} for i in range(len(docu-
ments))],
)

print(f"Added {collection.count()} documents to the collection")

# --- Querying with Semantic Search ---
def search(query: str, n_results: int = 3):
    """Perform semantic search on the collection."""
    query_embedding = embed_model.encode([query]).tolist()
    results = collection.query(
        query_embeddings=query_embedding,
        n_results=n_results,
        include=["documents", "distances", "metadatas"],
    )
    print(f"\nQuery: '{query}'")
    print(f"Top {n_results} results:")
    for i, (doc, dist) in enumerate(zip(results["documents"][0], results["distances"]
[0])):
        similarity = 1 - dist  # ChromaDB returns L2 distance by default
```

```
        print(f"  {i+1}. [{similarity:.3f}] {doc}")

# Test different queries
search("How do I train a model with labeled examples?")
search("What are neural network architectures for images?")
search("How do models learn from mistakes?")

# --- Filtering with Metadata ---
results = collection.query(
    query_embeddings=embed_model.encode(["neural network types"]).tolist(),
    n_results=5,
    where={"topic": "ml_fundamentals"},  # Metadata filter
)
print("\nFiltered results (ml_fundamentals only):")
for doc in results["documents"][0]:
    print(f"  • {doc}")
```

6.6 Advanced RAG Techniques

Hybrid Search (Semantic + Keyword)

```
# Advanced RAG: Hybrid search combining semantic and keyword matching
from langchain.retrievers import EnsembleRetriever
from langchain_community.retrievers import BM25Retriever
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

def create_hybrid_retriever(documents: list, chunks: list):
    """Create a hybrid retriever combining BM25 (keyword) and vector search."""

    # BM25 (keyword-based) retriever
    bm25_retriever = BM25Retriever.from_documents(chunks)
    bm25_retriever.k = 4

    # Vector (semantic) retriever
    embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
    vectorstore = Chroma.from_documents(chunks, embeddings)
    vector_retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

    # Combine with equal weighting
    ensemble_retriever = EnsembleRetriever(
        retrievers=[bm25_retriever, vector_retriever],
        weights=[0.4, 0.6],  # 40% keyword, 60% semantic
    )

    return ensemble_retriever
```

RAG with Reranking

```
# Advanced RAG: Reranking retrieved documents for higher relevance
from sentence_transformers import CrossEncoder

class RerankedRAG:
    """RAG with cross-encoder reranking for improved accuracy."""

    def __init__(self, retriever, llm):
        self.retriever = retriever
        self.llm = llm
        # Cross-encoder for reranking
        self.reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

    def query(self, question: str, initial_k: int = 10, final_k: int = 4) -> str:
        """Retrieve, rerank, then generate."""
        # Step 1: Retrieve more documents than we need
        docs = self.retriever.invoke(question)[:initial_k]

        # Step 2: Rerank using cross-encoder
        pairs = [(question, doc.page_content) for doc in docs]
        scores = self.reranker.predict(pairs)

        # Step 3: Take top-k after reranking
        ranked_indices = sorted(range(len(scores)), key=lambda i: scores[i], reverse=T
rue)
        top_docs = [docs[i] for i in ranked_indices[:final_k]]

        # Step 4: Generate answer with top reranked docs
        context = "\n\n".join(doc.page_content for doc in top_docs)
        prompt = f"Context:\n{context}\n\nQuestion: {question}\n\nAnswer:"
        return self.llm.invoke(prompt).content
```

🛠️ Project 5: Document Q&A System

Goal: Build a complete document Q&A system that can ingest PDFs, create a searchable knowledge base, and answer questions with source citations.

```python
# src/rag_app.py
"""Production-ready Document Q&A System with RAG."""

import os
from pathlib import Path
from dotenv import load_dotenv
from rich.console import Console
from rich.panel import Panel
from rich.prompt import Prompt
from rich.table import Table

load_dotenv()
console = Console()

class DocumentQA:
    """Complete document Q&A system with indexing, search, and generation."""

    def __init__(self, db_path: str = "./doc_qa_db"):
        from langchain_openai import ChatOpenAI, OpenAIEmbeddings
        from langchain_community.vectorstores import Chroma

        self.db_path = db_path
        self.embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
        self.llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
        self.vectorstore = None

    def ingest(self, path: str):
        """Ingest documents from a file or directory."""
        from langchain_community.document_loaders import (
            PyPDFLoader, TextLoader, DirectoryLoader,
        )
        from langchain.text_splitter import RecursiveCharacterTextSplitter
        from langchain_community.vectorstores import Chroma

        # Load documents based on type
        p = Path(path)
        if p.is_dir():
            loaders = {
                "**/*.pdf": PyPDFLoader,
                "**/*.txt": TextLoader,
                "**/*.md": TextLoader,
            }
            all_docs = []
            for glob_pattern, loader_cls in loaders.items():
                try:
                    loader = DirectoryLoader(str(p), glob=glob_pattern, loader_cls=loa
der_cls)
                    docs = loader.load()
                    all_docs.extend(docs)
                    if docs:
                        console.print(f"  Loaded {len(docs)} docs matching {glob_patte
rn}")
                except Exception:
                    pass
        elif p.suffix == ".pdf":
            all_docs = PyPDFLoader(str(p)).load()
        else:
            all_docs = TextLoader(str(p)).load()

        if not all_docs:
            console.print("[red]No documents found![/red]")
            return
```

```
        # Split into chunks
        splitter = RecursiveCharacterTextSplitter(
            chunk_size=1000,
            chunk_overlap=200,
            separators=["\n\n", "\n", ". ", " "],
        )
        chunks = splitter.split_documents(all_docs)

        # Create or update vector store
        self.vectorstore = Chroma.from_documents(
            documents=chunks,
            embedding=self.embeddings,
            persist_directory=self.db_path,
        )

        console.print(f"[green]✅ Indexed {len(chunks)} chunks from {len(all_docs)}
documents[/green]")

    def ask(self, question: str) -> dict:
        """Ask a question and get an answer with sources."""
        from langchain_community.vectorstores import Chroma

        if not self.vectorstore:
            self.vectorstore = Chroma(
                persist_directory=self.db_path,
                embedding_function=self.embeddings,
            )

        # Retrieve relevant chunks
        docs = self.vectorstore.similarity_search_with_score(question, k=4)

        # Build context
        context_parts = []
        sources = []
        for doc, score in docs:
            context_parts.append(doc.page_content)
            sources.append({
                "content": doc.page_content[:150],
                "source": doc.metadata.get("source", "unknown"),
                "relevance": round(1 - score, 3),
            })

        context = "\n\n---\n\n".join(context_parts)

        # Generate answer
        from langchain_core.prompts import ChatPromptTemplate
        prompt = ChatPromptTemplate.from_template(
            """Answer the question based ONLY on the following context. If the context
doesn't contain the answer, say "I cannot find this information in the provided docu-
ments."
Cite which parts of the context support your answer.

Context:
{context}

Question: {question}

Detailed Answer:"""
        )

        chain = prompt | self.llm
        response = chain.invoke({"context": context, "question": question})
```

```
        return {
            "answer": response.content,
            "sources": sources,
        }

    def interactive(self):
        """Run interactive Q&A session."""
        console.print(Panel(
            "[bold]📚 Document Q&A System[/bold]\n"
            "Commands: /ingest <path>, /quit",
            border_style="blue"
        ))

        while True:
            try:
                user_input = Prompt.ask("\n[bold green]Question[/bold green]")
            except (KeyboardInterrupt, EOFError):
                break

            if user_input.startswith("/quit"):
                break
            elif user_input.startswith("/ingest "):
                path = user_input.replace("/ingest ", "").strip()
                self.ingest(path)
                continue

            result = self.ask(user_input)
            console.print(Panel(result["answer"], title="Answer", bor-
der_style="green"))

            # Show sources
            table = Table(title="Sources", show_header=True)
            table.add_column("Relevance", style="cyan", justify="center")
            table.add_column("Source", style="yellow")
            table.add_column("Preview", style="white")
            for s in result["sources"]:
                table.add_row(str(s["relevance"]), s["source"], s["content"][:80] + ".
..")
            console.print(table)

if __name__ == "__main__":
    qa = DocumentQA()
    qa.interactive()
```

---

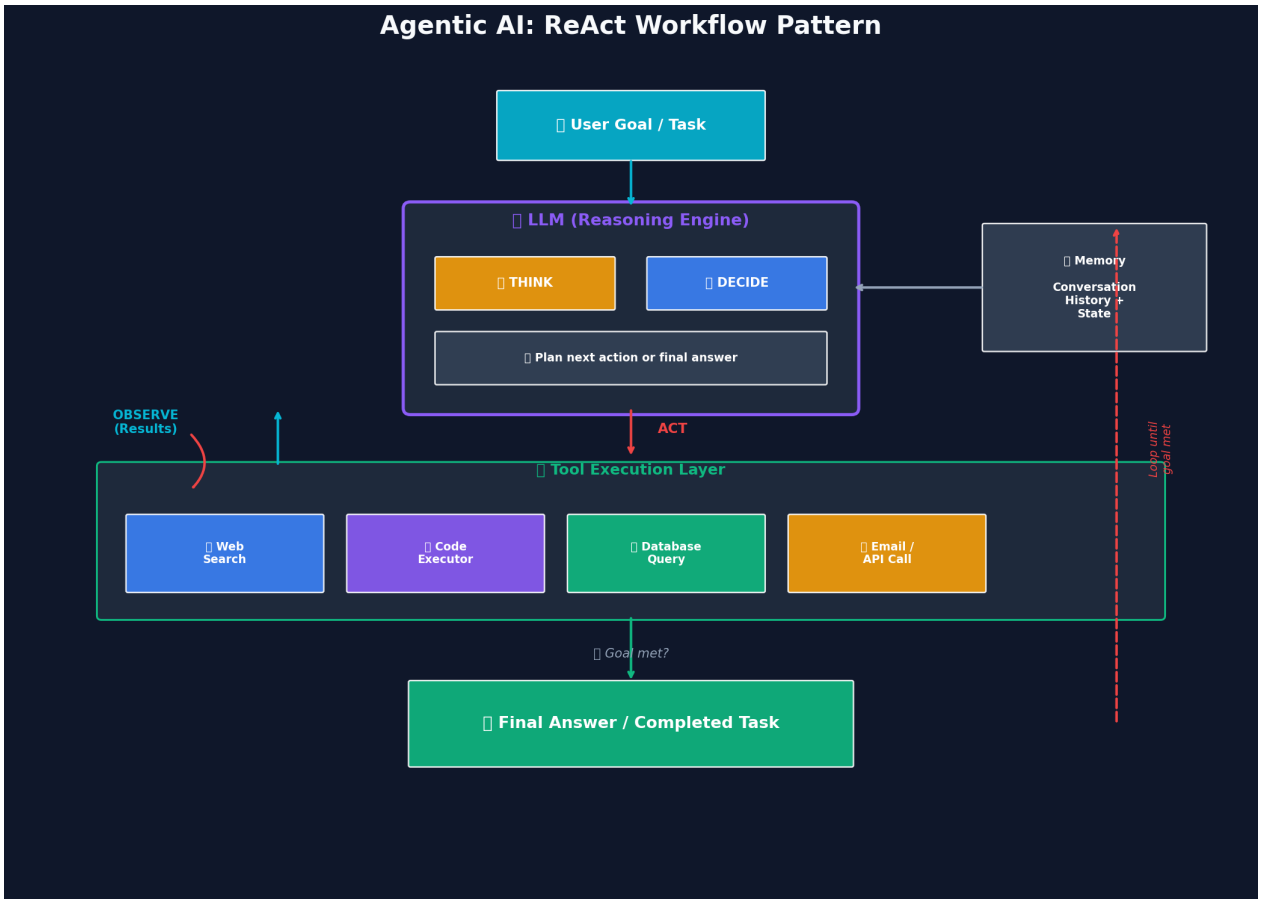
7. Agentic AI Patterns and Frameworks

Agentic AI represents the next evolution: systems that don't just respond to prompts but **autonomously reason, plan, and act** to accomplish complex goals.

7.1 What Makes an Agent?

An AI agent has four core capabilities:

1. **Reasoning**: Using an LLM as a "brain" to think through problems
2. **Planning**: Breaking complex goals into sequences of actionable steps
3. **Tool Use**: Calling external functions, APIs, and systems to gather information or take actions
4. **Memory**: Maintaining state across multiple steps and interactions



7.2 The ReAct Pattern — Reasoning and Acting

The **ReAct** (Reason + Act) pattern is the foundation of most agent systems. The agent iterates through a loop:

1. **Think**: Reason about the current state and what to do next
2. **Act**: Execute a tool or action
3. **Observe**: Process the result
4. **Repeat** until the goal is achieved

```python
# src/react_agent.py
"""Building a ReAct agent from scratch to understand the pattern."""

import json
import re
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()
client = OpenAI()

# ── Define Tools ──
def calculator(expression: str) -> str:
    """Safely evaluate a mathematical expression."""
    try:
        # Only allow safe mathematical operations
        allowed = set("0123456789+-*/.() ")
        if all(c in allowed for c in expression):
            result = eval(expression)
            return f"Result: {result}"
        return "Error: Invalid expression"
    except Exception as e:
        return f"Error: {e}"

def get_weather(city: str) -> str:
    """Simulated weather lookup (replace with real API in production)."""
    # In production, you'd call a weather API here
    weather_data = {
        "new york": "72°F, Partly Cloudy",
        "london": "59°F, Rainy",
        "tokyo": "68°F, Clear",
        "paris": "65°F, Overcast",
    }
    return weather_data.get(city.lower(), f"Weather data not available for {city}")

def search_knowledge(query: str) -> str:
    """Simulated knowledge search (replace with real search in production)."""
    knowledge = {
        "python creator": "Python was created by Guido van Rossum, first released in
1991.",
        "transformer paper": "The Transformer was introduced in 'Attention Is All You
Need' (2017) by Vaswani et al.",
        "rag definition": "RAG (Retrieval-Augmented Generation) combines information
retrieval with text generation.",
    }
    for key, value in knowledge.items():
        if key in query.lower():
            return value
    return f"No specific information found for: {query}"

# Tool registry
TOOLS = {
    "calculator": {
        "function": calculator,
        "description": "Evaluate mathematical expressions. Input: a math expression
string.",
    },
    "weather": {
        "function": get_weather,
        "description": "Get current weather for a city. Input: city name.",
    },
    "search": {
```

```
        "function": search_knowledge,
        "description": "Search a knowledge base for information. Input: search
query.",
    },
}

# ── The ReAct Agent ──
class ReActAgent:
    """A ReAct agent that reasons and uses tools to solve problems."""

    def __init__(self, tools: dict, max_iterations: int = 10):
        self.tools = tools
        self.max_iterations = max_iterations
        self.thought_history = []

    def _build_system_prompt(self) -> str:
        tool_descriptions = "\n".join(
            f"- {name}: {info['description']}" for name, info in self.tools.items()
        )
        return f"""You are a helpful AI agent that solves problems step by step.

You have access to these tools:
{tool_descriptions}

For each step, you MUST respond in EXACTLY this format:

Thought: [Your reasoning about what to do next]
Action: [tool_name]
Action Input: [input for the tool]

OR, if you have enough information to answer:

Thought: [Your final reasoning]
Final Answer: [Your complete answer to the user's question]

Always think step by step. Use tools when you need information you don't have."""

    def _parse_response(self, response: str) -> dict:
        """Parse the agent's response to extract thought, action, or final answer."""
        result = {"thought": "", "action": None, "action_input": None, "final_answer":
None}

        # Extract thought
        thought_match = re.search(r"Thought:\s*(.*?)(?=\nAction:|\nFinal Answer:|\Z)",
response, re.DOTALL)
        if thought_match:
            result["thought"] = thought_match.group(1).strip()

        # Check for final answer
        final_match = re.search(r"Final Answer:\s*(.*)", response, re.DOTALL)
        if final_match:
            result["final_answer"] = final_match.group(1).strip()
            return result

        # Extract action and input
        action_match = re.search(r"Action:\s*(.*)", response)
        input_match = re.search(r"Action Input:\s*(.*)", response)
        if action_match:
            result["action"] = action_match.group(1).strip()
        if input_match:
            result["action_input"] = input_match.group(1).strip()

        return result
```

```
    def run(self, user_query: str) -> str:
        """Execute the ReAct loop to answer a query."""
        messages = [
            {"role": "system", "content": self._build_system_prompt()},
            {"role": "user", "content": user_query},
        ]

        print(f"\n{'='*60}")
        print(f"🎯 Goal: {user_query}")
        print(f"{'='*60}")

        for i in range(self.max_iterations):
            # Get LLM response
            response = client.chat.completions.create(
                model="gpt-4o-mini",
                messages=messages,
                temperature=0,
                max_tokens=500,
            )
            agent_response = response.choices[0].message.content
            messages.append({"role": "assistant", "content": agent_response})

            # Parse the response
            parsed = self._parse_response(agent_response)
            print(f"\n--- Step {i+1} ---")
            print(f"💭 Thought: {parsed['thought']}")

            # Check if we have a final answer
            if parsed["final_answer"]:
                print(f"✅ Final Answer: {parsed['final_answer']}")
                return parsed["final_answer"]

            # Execute the tool
            if parsed["action"] and parsed["action"] in self.tools:
                tool = self.tools[parsed["action"]]
                print(f"🔧 Action: {parsed['action']}({parsed['action_input']})")
                observation = tool["function"](parsed["action_input"])
                print(f"👁️ Observation: {observation}")

                # Feed observation back to the agent
                messages.append({
                    "role": "user",
                    "content": f"Observation: {observation}",
                })
            else:
                print(f"⚠️ Unknown action: {parsed['action']}")
                messages.append({
                    "role": "user",
                    "content": f"Error: Tool '{parsed['action']}' not found. Avail-
able: {list(self.tools.keys())}",
                })

        return "Max iterations reached. Could not complete the task."

# ── Run the Agent ──
if __name__ == "__main__":
    agent = ReActAgent(TOOLS)

    # Test with various queries
    print(agent.run("What's 15% of 2847, and what's the weather like in Tokyo?"))
    print("\n" + "="*60 + "\n")
    print(agent.run("Who created Python and when was it first released?"))
```

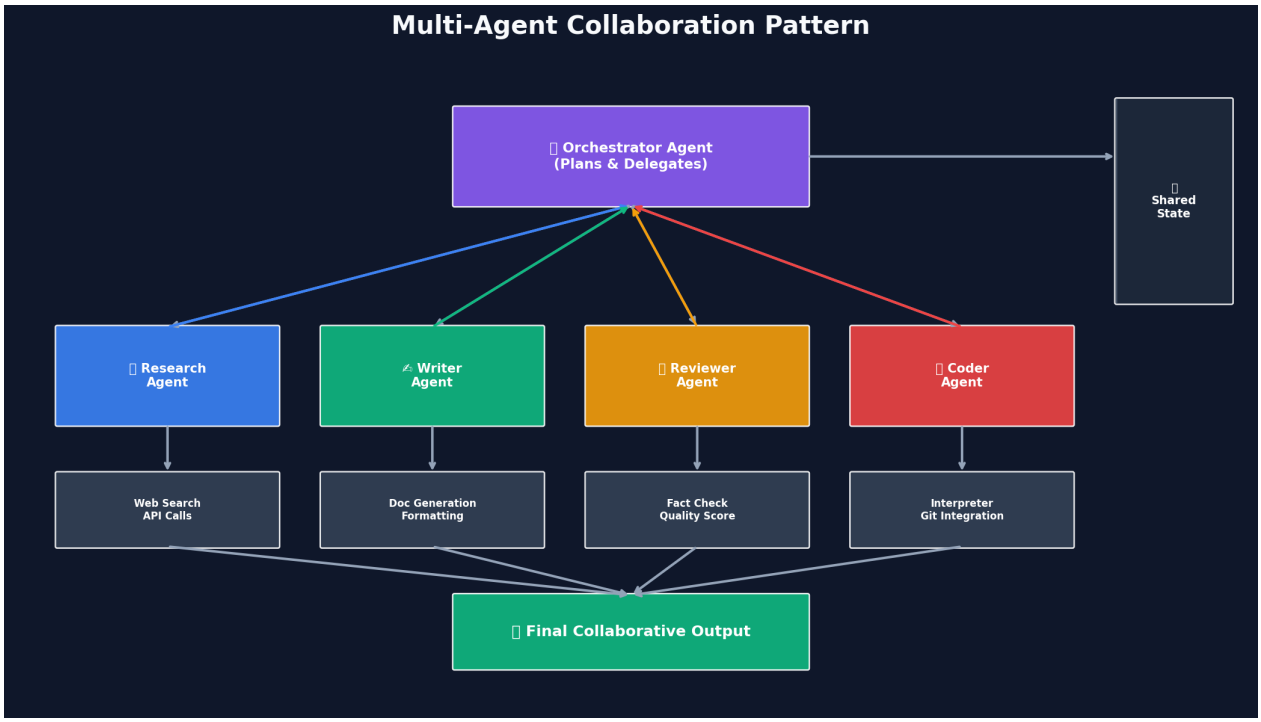
7.3 Building Agents with LangChain and LangGraph

LangChain provides production-ready agent tooling, and **LangGraph** enables complex stateful workflows.

```python
# src/langchain_agent.py
"""Building agents with LangChain's tool-calling framework."""

import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage
from langgraph.prebuilt import create_react_agent

load_dotenv()

# ── Define Tools Using LangChain's @tool Decorator ──
@tool
def calculate(expression: str) -> str:
    """Evaluate a mathematical expression. Use this for any math calculations.

    Args:
        expression: A mathematical expression like '2 + 2' or '15 * 0.2'
    """
    try:
        allowed = set("0123456789+-*/.() ")
        if all(c in allowed for c in expression):
            return str(eval(expression))
        return "Invalid expression"
    except Exception as e:
        return f"Error: {e}"

@tool
def get_current_date() -> str:
    """Get today's date. Use this when you need to know the current date."""
    from datetime import date
    return str(date.today())

@tool
def read_file(filepath: str) -> str:
    """Read the contents of a text file.

    Args:
        filepath: Path to the file to read
    """
    try:
        with open(filepath, "r") as f:
            content = f.read()
        return content[:5000]  # Limit to 5000 chars
    except Exception as e:
        return f"Error reading file: {e}"

@tool
def write_file(filepath: str, content: str) -> str:
    """Write content to a text file.

    Args:
        filepath: Path to the file to write
        content: Content to write to the file
    """
    try:
        os.makedirs(os.path.dirname(filepath) or ".", exist_ok=True)
        with open(filepath, "w") as f:
            f.write(content)
        return f"Successfully wrote {len(content)} characters to {filepath}"
    except Exception as e:
```

```
        return f"Error writing file: {e}"

# ── Create the Agent ──
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
tools = [calculate, get_current_date, read_file, write_file]

# LangGraph's create_react_agent handles the ReAct loop automatically
agent_executor = create_react_agent(llm, tools)

# ── Run the Agent ──
def run_agent(query: str):
    """Run the agent and display its steps."""
    print(f"\n🎯 Task: {query}")
    print("-" * 50)

    result = agent_executor.invoke(
        {"messages": [HumanMessage(content=query)]}
    )

    # Display the final response
    for message in result["messages"]:
        if hasattr(message, "content") and message.content:
            role = message.__class__.__name__.replace("Message", "")
            if role == "AI" and not hasattr(message, "tool_calls"):
                print(f"\n✅ Answer: {message.content}")
            elif role == "Tool":
                print(f"  🔧 Tool result: {message.content[:200]}")

if __name__ == "__main__":

run_agent("What is today's date? Then calculate how many days until the end of the
year.")
    run_agent("Write a Python hello world script to 'output/hello.py', then read it
back to verify.")
```

7.4 Multi-Agent Systems



Multi-agent systems assign different roles to specialized agents that collaborate:

```python
# src/multi_agent.py
"""Multi-agent system with specialized agents collaborating on tasks."""

from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()
client = OpenAI()

class SpecializedAgent:
    """An agent with a specific role and expertise."""

    def __init__(self, name: str, role: str, system_prompt: str):
        self.name = name
        self.role = role
        self.system_prompt = system_prompt

    def process(self, task: str, context: str = "") -> str:
        """Process a task with optional context from other agents."""
        messages = [
            {"role": "system", "content": self.system_prompt},
            {"role": "user", "content": f"Context from previous agents:\n{context}
\n\nTask: {task}"},
        ]

        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            temperature=0.7,
            max_tokens=1000,
        )
        return response.choices[0].message.content

class OrchestratorAgent:
    """Orchestrates multiple specialized agents to complete complex tasks."""

    def __init__(self):
        self.agents = {}
        self.execution_log = []

    def register_agent(self, agent: SpecializedAgent):
        """Register a specialized agent."""
        self.agents[agent.name] = agent

    def execute_pipeline(self, task: str, pipeline: list[dict]) -> dict:
        """Execute a pipeline of agent tasks.

        Args:
            task: The overall task description
            pipeline: List of dicts with 'agent', 'subtask', and optional 'depends_on'
        """
        results = {}
        print(f"\n{'='*60}")
        print(f"🎯 Orchestrating: {task}")
        print(f"{'='*60}")

        for step in pipeline:
            agent_name = step["agent"]
            subtask = step["subtask"]
            agent = self.agents.get(agent_name)

            if not agent:
```

```
                print(f"❌ Agent '{agent_name}' not found!")
                continue

            # Gather context from dependencies
            context = ""
            for dep in step.get("depends_on", []):
                if dep in results:
                    context += f"\n[{dep}'s output]:\n{results[dep]}\n"

            print(f"\n🤖 [{agent.name}] ({agent.role})")
            print(f"   Task: {subtask}")

            result = agent.process(subtask, context)
            results[agent_name] = result
            self.execution_log.append({
                "agent": agent_name,
                "task": subtask,
                "result": result[:200] + "...",
            })

            print(f"   ✅ Complete ({len(result)} chars)")

        return results

# ── Set Up Specialized Agents ──
researcher = SpecializedAgent(
    name="researcher",
    role="Research Analyst",
    system_prompt="""You are a thorough research analyst. When given a topic:
1. Identify 3-5 key aspects to investigate
2. Provide factual, well-structured findings
3. Note any areas of uncertainty
Be specific and cite concepts clearly.""",
)

writer = SpecializedAgent(
    name="writer",
    role="Technical Writer",
    system_prompt="""You are a skilled technical writer. Using the research provided:
1. Create clear, engaging prose
2. Organize with logical headings
3. Include practical examples
4. Keep the tone professional but accessible
Write in markdown format.""",
)

reviewer = SpecializedAgent(
    name="reviewer",
    role="Quality Reviewer",
    system_prompt="""You are a meticulous quality reviewer. Review the provided
content for:
1. Factual accuracy — flag anything questionable
2. Completeness — identify missing important points
3. Clarity — highlight confusing sections
4. Actionability — suggest specific improvements
Format your review as a structured assessment with ratings.""",
)

# ── Execute a Multi-Agent Pipeline ──
if __name__ == "__main__":
    orchestrator = OrchestratorAgent()
    orchestrator.register_agent(researcher)
    orchestrator.register_agent(writer)
```

```
    orchestrator.register_agent(reviewer)

    results = orchestrator.execute_pipeline(
        task="Create a brief guide on 'Getting Started with RAG in Python'",
        pipeline=[
            {
                "agent": "researcher",
                "subtask": "Research the key concepts, tools, and best practices for
building RAG systems in Python. Cover LangChain, LlamaIndex, vector databases, and
embedding models.",
            },
            {
                "agent": "writer",
                "subtask": "Write a concise beginner's guide based on the research.
Include a quick-start code example.",
                "depends_on": ["researcher"],
            },
            {
                "agent": "reviewer",
                "subtask": "Review the guide for accuracy, completeness, and clarity.
Suggest improvements.",
                "depends_on": ["researcher", "writer"],
            },
        ],
    )

    # Print the final guide and review
    print("\n" + "="*60)
    print("📄 FINAL GUIDE:")
    print("="*60)
    print(results.get("writer", "No output"))
    print("\n" + "="*60)
    print("📝 REVIEW:")
    print("="*60)
    print(results.get("reviewer", "No review"))
```

7.5 Agent Memory and State Management

```python
# src/agent_memory.py
"""Agent memory systems: short-term, long-term, and episodic memory."""

from datetime import datetime
from collections import deque
import json

class ConversationMemory:
    """Short-term memory: recent conversation turns."""

    def __init__(self, max_turns: int = 20):
        self.turns = deque(maxlen=max_turns)

    def add(self, role: str, content: str):
        self.turns.append({
            "role": role,
            "content": content,
            "timestamp": datetime.now().isoformat(),
        })

    def get_messages(self) -> list[dict]:
        return [{"role": t["role"], "content": t["content"]} for t in self.turns]

    def get_summary(self) -> str:
        return f"Conversation with {len(self.turns)} turns"

class SemanticMemory:
    """Long-term memory using vector storage for important facts."""

    def __init__(self):
        self.memories = []

    def store(self, content: str, metadata: dict = None):
        """Store a memory with optional metadata."""
        self.memories.append({
            "content": content,
            "metadata": metadata or {},
            "stored_at": datetime.now().isoformat(),
        })

    def search(self, query: str, top_k: int = 5) -> list[str]:
        """Simple keyword search (use vector search in production)."""
        scored = []
        query_words = set(query.lower().split())
        for mem in self.memories:
            mem_words = set(mem["content"].lower().split())
            overlap = len(query_words & mem_words)
            if overlap > 0:
                scored.append((overlap, mem["content"]))
        scored.sort(reverse=True)
        return [content for _, content in scored[:top_k]]

class EpisodicMemory:
    """Memory of complete task episodes (what worked, what didn't)."""

    def __init__(self):
        self.episodes = []

    def record_episode(self, task: str, steps: list[str], outcome: str, success: bool)
:
        """Record a complete task episode."""
        self.episodes.append({
```

```
            "task": task,
            "steps": steps,
            "outcome": outcome,
            "success": success,
            "timestamp": datetime.now().isoformat(),
        })

    def get_similar_episodes(self, task: str) -> list[dict]:
        """Find episodes similar to the current task."""
        task_words = set(task.lower().split())
        scored = []
        for ep in self.episodes:
            ep_words = set(ep["task"].lower().split())
            overlap = len(task_words & ep_words)
            if overlap > 0:
                scored.append((overlap, ep))
        scored.sort(reverse=True)
        return [ep for _, ep in scored[:3]]

    def save(self, filepath: str):
        with open(filepath, "w") as f:
            json.dump(self.episodes, f, indent=2)

    def load(self, filepath: str):
        with open(filepath, "r") as f:
            self.episodes = json.load(f)

class MemoryEnhancedAgent:
    """Agent with multiple memory systems."""

    def __init__(self):
        self.conversation = ConversationMemory()
        self.knowledge = SemanticMemory()
        self.episodes = EpisodicMemory()

    def process_with_memory(self, user_input: str) -> str:
        """Process input using all memory systems for context."""
        # Get relevant long-term memories
        relevant_knowledge = self.knowledge.search(user_input)

        # Get relevant past episodes
        similar_episodes = self.episodes.get_similar_episodes(user_input)

        # Build enriched context
        context_parts = []
        if relevant_knowledge:
            context_parts.append("Relevant knowledge:\n" + "\n".join(f"- {m}" for m in
relevant_knowledge))
        if similar_episodes:
            context_parts.append("Similar past tasks:\n" + "\n".join(
                f"- {ep['task']} → {'Success' if ep['success'] else 'Failed'}: {ep['ou
tcome'][:100]}"
                for ep in similar_episodes
            ))

        memory_context = "\n\n".join(context_parts) if context_parts else
"No relevant memories."

        # Store the interaction
        self.conversation.add("user", user_input)

        return memory_context  # In a real agent, this would be passed to the LLM
```

🛠️ Project 6: Research Assistant Agent

Goal: Build an agent that can research a topic using web search, synthesize findings, and produce a structured report.

```python
# src/agent.py
"""Research Assistant Agent — autonomously researches topics and writes reports."""

import os
import json
import requests
from datetime import datetime
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

# ── Tools for the Research Agent ──
def web_search(query: str) -> str:
    """Search the web for information (using a search API).

    In production, use SerpAPI, Tavily, or Brave Search API.
    This is a simplified version using DuckDuckGo.
    """
    try:
        # Using DuckDuckGo instant answer API (free, no key needed)
        resp = requests.get(
            "https://api.duckduckgo.com/",
            params={"q": query, "format": "json", "no_html": 1},
            timeout=10,
        )
        data = resp.json()
        results = []
        if data.get("Abstract"):
            results.append(f"Summary: {data['Abstract']}")
        for topic in data.get("RelatedTopics", [])[:5]:
            if isinstance(topic, dict) and "Text" in topic:
                results.append(topic["Text"])
        return "\n".join(results) if results else f"No results found for: {query}"
    except Exception as e:
        return f"Search error: {e}"

def take_notes(title: str, content: str) -> str:
    """Save research notes to a file."""
    os.makedirs("research_notes", exist_ok=True)
    filepath = f"research_notes/{title.replace(' ', '_').lower()}.md"
    with open(filepath, "a") as f:
        f.write(f"\n## {title}\n*{datetime.now().strftime('%Y-%m-%d %H:%M')}*\n\n{con-
tent}\n\n")
    return f"Notes saved to {filepath}"

def write_report(title: str, content: str) -> str:
    """Write the final research report."""
    os.makedirs("reports", exist_ok=True)
    filepath = f"reports/{title.replace(' ', '_').lower()}.md"
    with open(filepath, "w") as f:
        f.write(content)
    return f"Report saved to {filepath} ({len(content)} chars)"

RESEARCH_TOOLS = [
    {
        "type": "function",
        "function": {
            "name": "web_search",
            "description": "Search the web for information on a topic",
            "parameters": {
```

```
                "type": "object",
                "properties": {
                    "query": {"type": "string", "description": "Search query"}
                },
                "required": ["query"],
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "take_notes",
            "description": "Save research notes on a subtopic",
            "parameters": {
                "type": "object",
                "properties": {
                    "title": {"type": "string", "description": "Note title"},
                    "content": {"type": "string", "description": "Note content"},
                },
                "required": ["title", "content"],
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "write_report",
            "description": "Write the final research report in markdown",
            "parameters": {
                "type": "object",
                "properties": {
                    "title": {"type": "string", "description": "Report title"},
                    "content": {"type": "string", "description": "Full report in mark-
down"},
                },
                "required": ["title", "content"],
            },
        },
    },
]

TOOL_FUNCTIONS = {
    "web_search": web_search,
    "take_notes": take_notes,
    "write_report": write_report,
}

class ResearchAgent:
    """Autonomous research agent that investigates topics and produces reports."""

    def __init__(self):
        self.messages = []
        self.steps = []

    def research(self, topic: str, depth: str = "comprehensive") -> str:
        """Research a topic and produce a report."""
        system_prompt = f"""You are an expert research assistant. Your task is to
thoroughly
research a topic and produce a well-structured report.

Your approach:
1. PLAN: Break the topic into 3-5 key subtopics to investigate
2. RESEARCH: Use web_search to find information on each subtopic
```

```
3. NOTE: Use take_notes to record key findings for each subtopic
4. SYNTHESIZE: Once you have enough information, use write_report to create the final
report

The report should be {depth} and include:
- An executive summary
- Detailed sections for each subtopic
- Key findings and insights
- Conclusion with recommendations

Current date: {datetime.now().strftime('%Y-%m-%d')}"""

        self.messages = [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": f"Please research this topic thoroughly: {top-
ic}"},
        ]

        max_steps = 15
        for step in range(max_steps):
            response = client.chat.completions.create(
                model="gpt-4o-mini",
                messages=self.messages,
                tools=RESEARCH_TOOLS,
                temperature=0.3,
            )

            message = response.choices[0].message
            self.messages.append(message)

            # Check if the agent wants to call tools
            if message.tool_calls:
                for tool_call in message.tool_calls:
                    func_name = tool_call.function.name
                    func_args = json.loads(tool_call.function.arguments)

                    print(f"  🔧 [{func_name}] {json.dumps(func_args)[:100]}")

                    # Execute the tool
                    result = TOOL_FUNCTIONS[func_name](**func_args)
                    self.steps.append({"tool": func_name, "args": func_args, "result":
result[:200]})

                    # Add tool result to conversation
                    self.messages.append({
                        "role": "tool",
                        "tool_call_id": tool_call.id,
                        "content": result,
                    })
            else:
                # Agent is done (no more tool calls)
                final_message = message.content
                print(f"\n✅ Research complete in {step + 1} steps")
                return final_message

        return "Research completed (max steps reached)"

if __name__ == "__main__":
    from rich.console import Console
    console = Console()

    agent = ResearchAgent()
    console.print("[bold blue]🔬 Research Agent[/bold blue]")
```

```
    console.print("Starting research...\n")

    result = agent.research(
        topic="Current state of RAG techniques and best practices in 2026",
        depth="comprehensive",
    )

    console.print(f"\n[green]{result}[/green]")
```

---

8. Advanced Topics and Best Practices

8.1 Prompt Engineering Mastery

Effective prompt engineering is the highest-leverage skill in AI development.

The Prompt Engineering Hierarchy

```
Level 1: Basic Prompting      ├ "Summarize this text"
Level 2: Role Setting         ├ "You are an expert data scientist..."
Level 3: Structured Output    ├ "Return JSON with keys: summary, entities, sentiment"
Level 4: Few-Shot Learning    ├ "Here are 3 examples of the format I want..."
Level 5: Chain of Thought     ├ "Think step by step before answering"
Level 6: Self-Consistency     ├ Generate multiple answers and pick the most common
Level 7: Meta-Prompting       ├ Prompts that generate prompts
```

```python
# src/prompt_engineering.py
"""Advanced prompt engineering patterns."""

from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()
client = OpenAI()

def call_llm(messages: list[dict], **kwargs) -> str:
    response = client.chat.completions.create(
        model=kwargs.get("model", "gpt-4o-mini"),
        messages=messages,
        temperature=kwargs.get("temperature", 0),
        max_tokens=kwargs.get("max_tokens", 1000),
    )
    return response.choices[0].message.content

# ── Pattern 1: Chain of Thought ──
def chain_of_thought(question: str) -> str:
    """Force the model to reason step by step."""
    return call_llm([
        {"role": "system", "content": "You are a logical problem solver. "
         "Always think through problems step by step before giving your final answer.
"
         "Format: show your reasoning steps, then give the final answer."},
        {"role": "user", "content": question},
    ])

# ── Pattern 2: Few-Shot with Examples ──
def few_shot_classification(text: str) -> str:
    """Classify text using few-shot examples."""
    return call_llm([
        {"role": "system", "content": "Classify the following text into one of these
categories: "
         "TECHNICAL, BUSINESS, PERSONAL, NEWS. Return only the category name."},
        {"role": "user", "content": "How do I implement a binary search tree in Py-
thon?"},
        {"role": "assistant", "content": "TECHNICAL"},
        {"role": "user", "content": "Our Q3 revenue exceeded projections by 15%"},
        {"role": "assistant", "content": "BUSINESS"},
        {"role": "user", "content": "I'm planning a vacation to Italy next summer"},
        {"role": "assistant", "content": "PERSONAL"},
        {"role": "user", "content": text},
    ])

# ── Pattern 3: Self-Consistency (Multiple Samples) ──
def self_consistent_answer(question: str, n_samples: int = 5) -> str:
    """Generate multiple answers and return the most consistent one."""
    answers = []
    for _ in range(n_samples):
        answer = call_llm(
            [{"role": "user", "content": f"Answer concisely: {question}"}],
            temperature=0.7,
        )
        answers.append(answer)

    # Use another LLM call to synthesize the most consistent answer
    synthesis = call_llm([
        {"role": "system", "content": "You are given multiple answers to the same
question. "
         "Determine the most common/consistent answer and return it. "
```

```
         "If answers disagree, go with the majority."},
        {"role": "user", "content": f"Question: {question}\n\nAnswers:\n" +
         "\n".join(f"{i+1}. {a}" for i, a in enumerate(answers))},
    ])
    return synthesis

# ── Pattern 4: Structured Extraction ──
def extract_structured_data(text: str) -> str:
    """Extract structured information from unstructured text."""
    return call_llm([
        {"role": "system", "content":
"""Extract structured data from the text. Return valid JSON:
{
  "people": [{"name": "...", "role": "..."}],
  "organizations": ["..."],
  "dates": ["..."],
  "key_facts": ["..."],
  "sentiment": "positive|negative|neutral"
}"""},
        {"role": "user", "content": text},
    ], response_format={"type": "json_object"})

if __name__ == "__main__":
    # Test Chain of Thought
    print("=== Chain of Thought ===")
    print(chain_of_thought(
        "A store sells apples at $2 each. If I buy 3 apples and pay with a $10 bill, "
        "and the cashier gives me 2 quarters and some dollar bills as change, "
        "how many dollar bills did I receive?"
    ))

    print("\n=== Few-Shot Classification ===")
    print(few_shot_classification("The neural network achieved 95% accuracy on the
test set"))

    print("\n=== Structured Extraction ===")
    print(extract_structured_data(
        "Apple CEO Tim Cook announced at the September 2025 event in Cupertino that "
        "the new iPhone 17 Pro would feature an AI-powered camera system. "
        "Analysts from Goldman Sachs predicted strong holiday sales."
    ))
```

8.2 Error Handling and Resilience

Production AI applications must handle failures gracefully:

```python
# src/resilient_llm.py
"""Resilient LLM client with retries, fallbacks, and error handling."""

import time
import logging
from typing import Optional
from openai import OpenAI, APIError, RateLimitError, APITimeoutError
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class ResilientLLMClient:
    """LLM client with automatic retries, fallbacks, and circuit breaking."""

    def __init__(self):
        self.openai = OpenAI()
        self.anthropic = Anthropic()
        self.error_counts = {}
        self.circuit_breaker_threshold = 5

    def _is_circuit_open(self, provider: str) -> bool:
        """Check if too many errors have occurred for a provider."""
        return self.error_counts.get(provider, 0) >= self.circuit_breaker_threshold

    def _record_error(self, provider: str):
        self.error_counts[provider] = self.error_counts.get(provider, 0) + 1

    def _record_success(self, provider: str):
        self.error_counts[provider] = 0

    def chat(
        self,
        messages: list[dict],
        model: str = "gpt-4o-mini",
        max_retries: int = 3,
        fallback: bool = True,
        **kwargs,
    ) -> Optional[str]:
        """Send a chat request with retries and optional fallback."""

        # Try primary provider
        for attempt in range(max_retries):
            try:
                if self._is_circuit_open("openai"):
                    logger.warning("OpenAI circuit breaker OPEN, skipping to fall-
back")
                    break

                response = self.openai.chat.completions.create(
                    model=model,
                    messages=messages,
                    timeout=30,
                    **kwargs,
                )
                self._record_success("openai")
                return response.choices[0].message.content

            except RateLimitError:
                wait = 2 ** attempt  # Exponential backoff: 1s, 2s, 4s
```

```
                logger.warning(f"Rate limited. Waiting {wait}s (attempt {attempt + 1}/
{max_retries})")
                time.sleep(wait)

            except APITimeoutError:
                logger.warning(f"Timeout (attempt {attempt + 1}/{max_retries})")
                self._record_error("openai")

            except APIError as e:
                logger.error(f"API error: {e}")
                self._record_error("openai")
                if attempt == max_retries - 1:
                    break

        # Fallback to Anthropic
        if fallback:
            logger.info("Falling back to Anthropic Claude")
            try:
                # Convert messages format
                system = ""
                chat_msgs = []
                for msg in messages:
                    if msg["role"] == "system":
                        system = msg["content"]
                    else:
                        chat_msgs.append(msg)

                response = self.anthropic.messages.create(
                    model="claude-sonnet-4-20250514",
                    max_tokens=kwargs.get("max_tokens", 1024),
                    system=system,
                    messages=chat_msgs,
                )
                self._record_success("anthropic")
                return response.content[0].text

            except Exception as e:
                logger.error(f"Fallback also failed: {e}")
                self._record_error("anthropic")

        return None

if __name__ == "__main__":
    client = ResilientLLMClient()
    result = client.chat([
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "What is 2+2?"},
    ])
    print(f"Result: {result}")
```

8.3 Evaluation and Testing

```python
# src/evaluate_rag.py
"""Evaluation framework for RAG systems."""

from dataclasses import dataclass
from openai import OpenAI
from dotenv import load_dotenv
import json

load_dotenv()
client = OpenAI()

@dataclass
class EvalCase:
    question: str
    expected_answer: str
    context: str = ""

@dataclass
class EvalResult:
    question: str
    generated_answer: str
    expected_answer: str
    relevance_score: float
    accuracy_score: float
    completeness_score: float

def evaluate_answer(question: str, generated: str, expected: str) -> dict:
    """Use an LLM to evaluate the quality of a generated answer."""
    eval_prompt = f"""Evaluate the quality of a generated answer compared to the
expected answer.

Question: {question}
Expected Answer: {expected}
Generated Answer: {generated}

Score each dimension from 0.0 to 1.0:
1. relevance: Does the answer address the question?
2. accuracy: Is the answer factually correct compared to the expected answer?
3. completeness: Does the answer cover all key points from the expected answer?

Return JSON: {{"relevance": 0.0, "accuracy": 0.0, "completeness": 0.0, "explanation":
"..."}}"""

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": eval_prompt}],
        response_format={"type": "json_object"},
        temperature=0,
    )

    return json.loads(response.choices[0].message.content)

def run_evaluation(rag_function, test_cases: list[EvalCase]) -> list[EvalResult]:
    """Run evaluation across a set of test cases."""
    results = []
    for case in test_cases:
        generated = rag_function(case.question)
        scores = evaluate_answer(case.question, generated, case.expected_answer)

        result = EvalResult(
            question=case.question,
            generated_answer=generated,
```

```
            expected_answer=case.expected_answer,
            relevance_score=scores["relevance"],
            accuracy_score=scores["accuracy"],
            completeness_score=scores["completeness"],
        )
        results.append(result)

    # Summary
    avg_relevance = sum(r.relevance_score for r in results) / len(results)
    avg_accuracy = sum(r.accuracy_score for r in results) / len(results)
    avg_completeness = sum(r.completeness_score for r in results) / len(results)

    print(f"\n📊 Evaluation Results ({len(results)} test cases)")
    print(f"  Relevance:    {avg_relevance:.2f}")
    print(f"  Accuracy:     {avg_accuracy:.2f}")
    print(f"  Completeness: {avg_completeness:.2f}")
    print(f"  Overall:      {(avg_relevance + avg_accuracy + avg_completeness) / 3:.
2f}")

    return results
```

8.4 Performance Optimization

```python
# src/optimization.py
"""Performance optimization techniques for AI applications."""

import asyncio
import time
from openai import AsyncOpenAI
from dotenv import load_dotenv

load_dotenv()

# ── Async Parallel Requests ──
async def parallel_completions(prompts: list[str], model: str = "gpt-4o-mini") ->
list[str]:
    """Send multiple LLM requests in parallel using async."""
    client = AsyncOpenAI()

    async def single_request(prompt: str) -> str:
        response = await client.chat.completions.create(
            model=model,
            messages=[{"role": "user", "content": prompt}],
            max_tokens=200,
        )
        return response.choices[0].message.content

    # Run all requests concurrently
    tasks = [single_request(p) for p in prompts]
    results = await asyncio.gather(*tasks)
    return results

# ── Caching Layer ──
import hashlib
import json
import os

class LLMCache:
    """Simple file-based cache for LLM responses."""

    def __init__(self, cache_dir: str = ".llm_cache"):
        self.cache_dir = cache_dir
        os.makedirs(cache_dir, exist_ok=True)
        self.hits = 0
        self.misses = 0

    def _hash_key(self, messages: list[dict], model: str, **kwargs) -> str:
        key = json.dumps({"messages": messages, "model": model, **kwargs}, sort_keys=T
rue)
        return hashlib.sha256(key.encode()).hexdigest()

    def get(self, messages: list[dict], model: str, **kwargs) -> str | None:
        key = self._hash_key(messages, model, **kwargs)
        path = os.path.join(self.cache_dir, f"{key}.json")
        if os.path.exists(path):
            self.hits += 1
            with open(path) as f:
                return json.load(f)["response"]
        self.misses += 1
        return None

    def set(self, messages: list[dict], model: str, response: str, **kwargs):
        key = self._hash_key(messages, model, **kwargs)
        path = os.path.join(self.cache_dir, f"{key}.json")
        with open(path, "w") as f:
```

```
            json.dump({"response": response, "model": model}, f)

    def stats(self) -> dict:
        total = self.hits + self.misses
        return {
            "hits": self.hits,
            "misses": self.misses,
            "hit_rate": f"{self.hits / total * 100:.1f}%" if total > 0 else "N/A",
        }

# ── Usage Example ──
if __name__ == "__main__":
    # Parallel completions
    prompts = [
        "What is Python?",
        "What is JavaScript?",
        "What is Rust?",
        "What is Go?",
        "What is TypeScript?",
    ]

    start = time.time()
    results = asyncio.run(parallel_completions(prompts))
    elapsed = time.time() - start

    print(f"Completed {len(prompts)} requests in {elapsed:.1f}s (parallel)")
    for prompt, result in zip(prompts, results):
        print(f"\n  Q: {prompt}")
        print(f"  A: {result[:100]}...")
```

8.5 Security Best Practices

```python
# src/security.py
"""Security best practices for AI applications."""

import re
import os

# ── 1. Prompt Injection Defense ──
def sanitize_user_input(user_input: str) -> str:
    """Basic sanitization to reduce prompt injection risks."""
    # Remove common injection patterns
    dangerous_patterns = [
        r"ignore\s+(previous|above|all)\s+(instructions|prompts)",
        r"you\s+are\s+now\s+",
        r"system\s*:\s*",
        r"<\|.*?\|>",
        r"\[INST\]",
    ]
    sanitized = user_input
    for pattern in dangerous_patterns:
        sanitized = re.sub(pattern, "[filtered]", sanitized, flags=re.IGNORECASE)
    return sanitized

# ── 2. Output Validation ──
def validate_llm_output(output: str, max_length: int = 5000) -> dict:
    """Validate LLM output before presenting to users."""
    issues = []
    if len(output) > max_length:
        output = output[:max_length]
        issues.append("truncated")

    # Check for potential data leakage patterns
    sensitive_patterns = [
        (r"sk-[a-zA-Z0-9]{20,}", "API key detected"),
        (r"\b\d{3}-\d{2}-\d{4}\b", "Possible SSN pattern"),
        (r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}", "Email detected"),
    ]
    for pattern, description in sensitive_patterns:
        if re.search(pattern, output):
            issues.append(description)
            output = re.sub(pattern, "[REDACTED]", output)

    return {"content": output, "issues": issues, "safe": len(issues) == 0}

# ── 3. API Key Management ──
def get_api_key(service: str) -> str:
    """Securely retrieve API keys from environment variables."""
    key_name = f"{service.upper()}_API_KEY"
    key = os.environ.get(key_name)
    if not key:
        raise EnvironmentError(
            f"Missing {key_name}. Set it in your .env file or environment variables. "
            f"NEVER hardcode API keys in source code."
        )
    return key
```

🛠️ Project 7: AI-Powered Code Review Agent

Goal: Build an agent that reviews Python code for quality, security, and best practices.

```python
# src/code_reviewer.py
"""AI-Powered Code Review Agent."""

import os
import json
from openai import OpenAI
from dotenv import load_dotenv
from pathlib import Path

load_dotenv()
client = OpenAI()

class CodeReviewAgent:
    """Autonomous code review agent that analyzes Python files."""

    REVIEW_CATEGORIES = [
        "code_quality",
        "security_vulnerabilities",
        "performance",
        "best_practices",
        "documentation",
        "error_handling",
    ]

    def review_file(self, filepath: str) -> dict:
        """Review a single Python file."""
        code = Path(filepath).read_text()

        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[
                {"role": "system", "content": """You are an expert Python code review-
er.
Analyze the code and provide a detailed review. Return JSON with this structure:
{
    "overall_score": 0-10,
    "summary": "Brief overall assessment",
    "issues": [
        {
            "severity": "critical|warning|info",
            "category": "security|performance|quality|style|documentation",
            "line": null or line_number,
            "description": "What's wrong",
            "suggestion": "How to fix it",
            "code_fix": "Corrected code snippet if applicable"
        }
    ],
    "strengths": ["Things done well"],
    "recommendations": ["Top 3 improvements to prioritize"]
}"""},
                {"role": "user", "content": f"Review this Python code:
\n\n```python\n{code}\n```"},
            ],
            response_format={"type": "json_object"},
            temperature=0,
        )

        review = json.loads(response.choices[0].message.content)
        review["file"] = filepath
        return review

    def review_directory(self, directory: str) -> list[dict]:
```

```
        """Review all Python files in a directory."""
        reviews = []
        for filepath in Path(directory).rglob("*.py"):
            if "__pycache__" in str(filepath) or ".pixi" in str(filepath):
                continue
            print(f"  Reviewing {filepath}...")
            review = self.review_file(str(filepath))
            reviews.append(review)
        return reviews

    def generate_report(self, reviews: list[dict]) -> str:
        """Generate a markdown report from all reviews."""
        report = "# Code Review Report\n\n"
        report += f"**Files Reviewed:** {len(reviews)}\n\n"

        total_issues = sum(len(r.get("issues", [])) for r in reviews)
        critical = sum(1 for r in reviews for i in r.get("issues", []) if
i["severity"] == "critical")
        warnings = sum(1 for r in reviews for i in r.get("issues", []) if
i["severity"] == "warning")

        report += f"**Total Issues:** {total_issues} ({critical} critical, {warnings}
warnings)\n\n"

        for review in reviews:
            score = review.get("overall_score", "N/A")
            report += f"## {review['file']} — Score: {score}/10\n\n"
            report += f"{review.get('summary', '')}\n\n"

            if review.get("issues"):
                report += "### Issues\n\n"
                for issue in review["issues"]:
                    emoji = {"critical": "🔴", "warning": "🟡", "info": "🔵"}.get(issu
e["severity"], "⚪")
                    report += f"- {emoji} **{issue['severity'].upper()}** [{issue['cat
egory']}]: {issue['description']}\n"
                    report += f"  - *Fix:* {issue['suggestion']}\n"
                    if issue.get("code_fix"):
                        report += f"  ```python\n  {issue['code_fix']}\n  ```\n"
                report += "\n"

            if review.get("strengths"):
                report += "### Strengths\n"
                for s in review["strengths"]:
                    report += f"- ✅ {s}\n"
                report += "\n"

        return report

if __name__ == "__main__":
    from rich.console import Console
    console = Console()

    reviewer = CodeReviewAgent()
    console.print("[bold]🔍 AI Code Review Agent[/bold]\n")

    # Review the src directory
    reviews = reviewer.review_directory("src/")
    report = reviewer.generate_report(reviews)

    # Save report
    with open("code_review_report.md", "w") as f:
        f.write(report)
```

```
    console.print(f"[green]✅ Review complete! Report saved to code_review_report.md[/
green]")
```

---

9. Capstone: Full-Stack AI Application

9.1 Project Architecture

This capstone combines everything: a RAG-powered document Q&A system with an agent layer for advanced tasks, exposed through a web API.

```
┌──────────────────────────────────────────────┐
│                  FastAPI Server              │
├──────────┬──────────┬──────────────────────┤
│ /chat    │ /ingest  │ /agent               │
├──────────┴──────────┴──────────────────────┤
│            Application Layer                 │
│   ┌──────────┐   ┌──────────┐   ┌──────────┐ │
│   │  RAG     │   │  Agent   │   │  Memory  │ │
│   │  Engine  │   │  Engine  │   │  Manager │ │
│   └──────────┘   └──────────┘   └──────────┘ │
│        │              │              │       │
│   ┌────▼──────────────▼──────────────▼────┐  │
│   │          LLM Provider Layer           │  │
│   │   (OpenAI / Anthropic / Local)        │  │
│   └──────────────────────────────────────┘  │
│                     │                        │
│   ┌─────────────────▼────────────────────┐  │
│   │          ChromaDB Vector Store        │  │
│   └──────────────────────────────────────┘  │
└──────────────────────────────────────────────┘
```

9.2 The Pixi Configuration

```toml
# pixi.toml for the capstone project
[project]
name = "ai-capstone"
version = "1.0.0"
description = "Full-stack AI application with RAG and Agents"
channels = ["conda-forge"]
platforms = ["linux-64", "osx-arm64", "osx-64", "win-64"]

[dependencies]
python = ">=3.11,<3.13"

# LLM Providers
openai = ">=1.40"
anthropic = ">=0.34"

# RAG Stack
langchain = ">=0.3"
langchain-openai = ">=0.2"
langchain-community = ">=0.3"
chromadb = ">=0.5"
sentence-transformers = ">=3.0"

# Web Framework
fastapi = ">=0.115"
uvicorn = ">=0.30"
python-multipart = ">=0.0.9"

# Document Processing
pypdf = ">=4.0"
beautifulsoup4 = ">=4.12"
python-dotenv = ">=1.0"

# Utilities
rich = ">=13.0"
pydantic = ">=2.8"

[tasks]
serve = "uvicorn src.app:app --reload --port 8000"
test = "python -m pytest tests/ -v"
ingest = "python src/ingest.py"
```

9.3 The Application Code

```python
# src/app.py
"""FastAPI application combining RAG and Agent capabilities."""

from fastapi import FastAPI, UploadFile, File, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from typing import Optional
import os
import shutil
from dotenv import load_dotenv

load_dotenv()

app = FastAPI(
    title="AI Knowledge Assistant",
    description="RAG-powered Q&A with Agent capabilities",
    version="1.0.0",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# ── Request/Response Models ──
class ChatRequest(BaseModel):
    question: str
    use_rag: bool = True
    model: str = "gpt-4o-mini"

class ChatResponse(BaseModel):
    answer: str
    sources: list[dict] = []
    tokens_used: int = 0

class AgentRequest(BaseModel):
    task: str
    max_steps: int = 10

class AgentResponse(BaseModel):
    result: str
    steps: list[dict] = []

# ── Initialize Components ──
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

VECTOR_DB_PATH = "./capstone_vectordb"
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

def get_vectorstore():
    """Get or create the vector store."""
    if os.path.exists(VECTOR_DB_PATH):
        return Chroma(persist_directory=VECTOR_DB_PATH, embedding_function=embeddings)
    return None

# ── Endpoints ──
```

```
@app.get("/health")
def health_check():
    return {"status": "healthy", "has_index": os.path.exists(VECTOR_DB_PATH)}

@app.post("/ingest")
async def ingest_document(file: UploadFile = File(...)):
    """Ingest a document into the knowledge base."""
    from langchain_community.document_loaders import PyPDFLoader, TextLoader
    from langchain.text_splitter import RecursiveCharacterTextSplitter

    # Save uploaded file
    upload_dir = "uploads"
    os.makedirs(upload_dir, exist_ok=True)
    filepath = os.path.join(upload_dir, file.filename)
    with open(filepath, "wb") as f:
        shutil.copyfileobj(file.file, f)

    # Load and split
    if filepath.endswith(".pdf"):
        loader = PyPDFLoader(filepath)
    else:
        loader = TextLoader(filepath)

    docs = loader.load()
    splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
    chunks = splitter.split_documents(docs)

    # Add to vector store
    vectorstore = get_vectorstore()
    if vectorstore:
        vectorstore.add_documents(chunks)
    else:
        Chroma.from_documents(chunks, embeddings, persist_directory=VECTOR_DB_PATH)

    return {"message": f"Ingested {len(chunks)} chunks from {file.filename}"}

@app.post("/chat", response_model=ChatResponse)
async def chat(request: ChatRequest):
    """Ask a question, optionally using RAG."""
    llm = ChatOpenAI(model=request.model, temperature=0)

    if request.use_rag:
        vectorstore = get_vectorstore()
        if not vectorstore:
            raise HTTPException(status_code=400, detail="No documents indexed yet. Up-
load documents first.")

        retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
        docs = retriever.invoke(request.question)

        context = "\n\n".join(doc.page_content for doc in docs)
        prompt = ChatPromptTemplate.from_template(
            "Answer based on this context:\n\n{context}\n\nQuestion: {question}\n\nAn-
swer:"
        )
        chain = prompt | llm | StrOutputParser()
        answer = chain.invoke({"context": context, "question": request.question})

        sources = [{"content": d.page_content[:200], "source":
d.metadata.get("source", "")} for d in docs]
        return ChatResponse(answer=answer, sources=sources)
    else:
        response = llm.invoke(request.question)
```

```
        return ChatResponse(answer=response.content)

@app.post("/agent", response_model=AgentResponse)
async def run_agent(request: AgentRequest):
    """Run an agent to complete a complex task."""
    from langchain_openai import ChatOpenAI
    from langchain_core.tools import tool
    from langgraph.prebuilt import create_react_agent

    @tool
    def search_knowledge_base(query: str) -> str:
        """Search the indexed documents for information."""
        vectorstore = get_vectorstore()
        if not vectorstore:
            return "No documents indexed."
        docs = vectorstore.similarity_search(query, k=3)
        return "\n".join(d.page_content for d in docs)

    @tool
    def calculate(expression: str) -> str:
        """Evaluate a math expression."""
        try:
            return str(eval(expression))
        except:
            return "Invalid expression"

    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    agent = create_react_agent(llm, [search_knowledge_base, calculate])

    from langchain_core.messages import HumanMessage
    result = agent.invoke({"messages": [HumanMessage(content=request.task)]})

    steps = []
    final_answer = ""
    for msg in result["messages"]:
        if hasattr(msg, "tool_calls") and msg.tool_calls:
            for tc in msg.tool_calls:
                steps.append({"tool": tc["name"], "input": str(tc["args"])})
        if hasattr(msg, "content") and msg.content and msg.__class__.__name__ == "AIMe
ssage":
            final_answer = msg.content

    return AgentResponse(result=final_answer, steps=steps)
```

9.4 Running the Capstone

```
# Install dependencies
pixi install

# Start the server
pixi run serve

# In another terminal — test the API:
# 1. Upload a document
curl -X POST http://localhost:8000/ingest \
  -F "file=@data/sample_document.pdf"

# 2. Ask a question with RAG
curl -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"question": "What are the key topics in the document?", "use_rag": true}'

# 3. Run an agent task
curl -X POST http://localhost:8000/agent \
  -H "Content-Type: application/json" \
  -d '{"task": "Search the knowledge base for information about AI and summarize
it."}'
```

---

10. Production Deployment Patterns

10.1 Containerizing Your AI Application

Deploying AI applications to production requires containerization. Here’s a production-ready Dockerfile:

```dockerfile
# Dockerfile for the AI Knowledge Assistant
FROM python:3.12-slim

# Install system dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Install Pixi
RUN curl -fsSL https://pixi.sh/install.sh | sh

WORKDIR /app

# Copy project files
COPY pixi.toml pixi.lock ./
RUN /root/.pixi/bin/pixi install --locked

COPY src/ src/
COPY .env .env

# Expose the API port
EXPOSE 8000

# Health check
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
    CMD curl -f http://localhost:8000/health || exit 1

# Run with Pixi
CMD ["/root/.pixi/bin/pixi", "run", "serve"]
```

Docker Compose for the full stack:

```yaml
# docker-compose.yml
version: '3.8'

services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
    volumes:
      - vectordb_data:/app/capstone_vectordb
      - upload_data:/app/uploads
    restart: unless-stopped
    deploy:
      resources:
        limits:
          memory: 4G

volumes:
  vectordb_data:
  upload_data:
```

10.2 Monitoring and Observability

In production, you need to monitor your AI application's performance, costs, and quality. Here's a comprehensive monitoring setup:

```python
# src/monitoring.py
"""Production monitoring for AI applications."""

import time
import logging
from dataclasses import dataclass, field
from datetime import datetime
from typing import Optional
from collections import defaultdict

logger = logging.getLogger(__name__)

@dataclass
class LLMCallMetrics:
    """Metrics for a single LLM API call."""
    model: str
    input_tokens: int
    output_tokens: int
    latency_ms: float
    success: bool
    error: Optional[str] = None
    timestamp: str = field(default_factory=lambda: datetime.now().isoformat())
    cost_usd: float = 0.0

class AIMonitor:
    """Monitor AI application performance, costs, and reliability."""

    # Approximate pricing per 1M tokens
    PRICING = {
        "gpt-4o": {"input": 2.50, "output": 10.00},
        "gpt-4o-mini": {"input": 0.15, "output": 0.60},
        "claude-sonnet-4-20250514": {"input": 3.00, "output": 15.00},
    }

    def __init__(self):
        self.metrics: list[LLMCallMetrics] = []
        self.error_counts: dict[str, int] = defaultdict(int)
        self.start_time = datetime.now()

    def record_call(self, metric: LLMCallMetrics):
        """Record metrics for an LLM call."""
        # Calculate cost
        pricing = self.PRICING.get(metric.model, {"input": 1.0, "output": 2.0})
        metric.cost_usd = (
            (metric.input_tokens / 1_000_000) * pricing["input"] +
            (metric.output_tokens / 1_000_000) * pricing["output"]
        )

        self.metrics.append(metric)

        if not metric.success:
            self.error_counts[metric.error or "unknown"] += 1
            logger.warning(f"LLM call failed: {metric.error} (model: {metric.model})")

    def get_summary(self) -> dict:
        """Get a summary of all recorded metrics."""
        if not self.metrics:
            return {"message": "No metrics recorded yet"}

        total_calls = len(self.metrics)
        successful = sum(1 for m in self.metrics if m.success)
        total_cost = sum(m.cost_usd for m in self.metrics)
```

```
        total_input_tokens = sum(m.input_tokens for m in self.metrics)
        total_output_tokens = sum(m.output_tokens for m in self.metrics)
        avg_latency = sum(m.latency_ms for m in self.metrics) / total_calls

        # Per-model breakdown
        model_stats = defaultdict(lambda: {"calls": 0, "cost": 0.0, "tokens": 0})
        for m in self.metrics:
            model_stats[m.model]["calls"] += 1
            model_stats[m.model]["cost"] += m.cost_usd
            model_stats[m.model]["tokens"] += m.input_tokens + m.output_tokens

        return {
            "total_calls": total_calls,
            "success_rate": f"{(successful / total_calls) * 100:.1f}%",
            "total_cost_usd": round(total_cost, 4),
            "total_input_tokens": total_input_tokens,
            "total_output_tokens": total_output_tokens,
            "avg_latency_ms": round(avg_latency, 1),
            "p95_latency_ms": round(sorted(m.latency_ms for m in self.metrics)[int(tot
al_calls * 0.95)], 1),
            "error_breakdown": dict(self.error_counts),
            "model_breakdown": dict(model_stats),
            "uptime_seconds": (datetime.now() - self.start_time).total_seconds(),
        }

    def alert_check(self) -> list[str]:
        """Check for conditions that should trigger alerts."""
        alerts = []
        summary = self.get_summary()

        if isinstance(summary.get("success_rate"), str):
            rate = float(summary["success_rate"].rstrip("%"))
            if rate < 95:
                alerts.append(f"⚠️ Success rate below 95%: {summary['success_rate']}")

        if summary.get("avg_latency_ms", 0) > 5000:
            alerts.append(f"⚠️ High average latency: {summary['avg_latency_ms']}ms")

        if summary.get("total_cost_usd", 0) > 10:
            alerts.append(f"💰 Cost alert: ${summary['total_cost_usd']:.
2f} spent this session")

        return alerts

# Context manager for tracking LLM calls
class track_llm_call:
    """Context manager to automatically track LLM call metrics."""

    def __init__(self, monitor: AIMonitor, model: str):
        self.monitor = monitor
        self.model = model
        self.start_time = None

    def __enter__(self):
        self.start_time = time.time()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        latency = (time.time() - self.start_time) * 1000
        self.monitor.record_call(LLMCallMetrics(
            model=self.model,
            input_tokens=getattr(self, "input_tokens", 0),
            output_tokens=getattr(self, "output_tokens", 0),
```

```
            latency_ms=latency,
            success=exc_type is None,
            error=str(exc_val) if exc_val else None,
        ))
        return False  # Don't suppress exceptions
```

10.3 Cost Optimization Strategies

Managing costs is critical for production AI applications. Here are proven strategies:

```python
# src/cost_optimization.py
"""Cost optimization strategies for AI applications."""

from functools import lru_cache
import hashlib
import json
import os
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()
client = OpenAI()

# ── Strategy 1: Tiered Model Routing ──
class ModelRouter:
    """Route queries to the most cost-effective model based on complexity."""

    MODELS = {
        "simple": {"name": "gpt-4o-mini", "cost_per_1k": 0.00015},
        "medium": {"name": "gpt-4o-mini", "cost_per_1k": 0.00015},
        "complex": {"name": "gpt-4o", "cost_per_1k": 0.0025},
    }

    def classify_complexity(self, query: str) -> str:
        """Estimate query complexity to choose the right model."""
        # Simple heuristics (use a classifier in production)
        word_count = len(query.split())
        has_code = any(kw in query.lower() for kw in ["code", "implement", "debug", "f
unction"])
        needs_reasoning = any(kw in query.lower() for kw in ["analyze", "compare", "ex
plain why", "evaluate"])

        if word_count < 20 and not has_code and not needs_reasoning:
            return "simple"
        elif has_code or needs_reasoning:
            return "complex"
        return "medium"

    def route(self, query: str) -> str:
        """Get the appropriate model for a query."""
        complexity = self.classify_complexity(query)
        return self.MODELS[complexity]["name"]

# ── Strategy 2: Prompt Compression ──
def compress_context(context: str, max_tokens: int = 2000) -> str:
    """Compress long contexts to reduce token usage.

    In production, use libraries like LLMLingua for better compression.
    This is a simplified approach using extractive summarization.
    """
    sentences = context.split(". ")
    if len(sentences) <= 5:
        return context

    # Use a cheap model to summarize
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "system",
                "content": f"Compress the following text to under {max_tokens} tokens
while preserving all key facts. Remove redundancy but keep specifics.",
```

```
            },
            {"role": "user", "content": context},
        ],
        max_tokens=max_tokens,
    )
    return response.choices[0].message.content

# ── Strategy 3: Semantic Caching ──
class SemanticCache:
    """Cache LLM responses based on semantic similarity of queries."""

    def __init__(self, similarity_threshold: float = 0.92):
        self.cache = {}  # {query_hash: (query, response)}
        self.threshold = similarity_threshold

    def _get_embedding(self, text: str) -> list[float]:
        response = client.embeddings.create(
            model="text-embedding-3-small",
            input=text,
        )
        return response.data[0].embedding

    def _cosine_sim(self, a: list[float], b: list[float]) -> float:
        import numpy as np
        a, b = np.array(a), np.array(b)
        return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

    def get(self, query: str) -> str | None:
        """Check if a semantically similar query has been cached."""
        if not self.cache:
            return None

        query_emb = self._get_embedding(query)
        for cached_query, (cached_emb, response) in self.cache.items():
            if self._cosine_sim(query_emb, cached_emb) >= self.threshold:
                return response
        return None

    def set(self, query: str, response: str):
        """Cache a query-response pair."""
        query_emb = self._get_embedding(query)
        key = hashlib.md5(query.encode()).hexdigest()
        self.cache[key] = (query_emb, response)
```

10.4 Handling Long Documents — Chunking Strategies

The way you split documents into chunks has a massive impact on RAG quality. Here's a comprehensive look at strategies:

```python
# src/chunking_strategies.py
"""Advanced document chunking strategies for RAG."""

from langchain.text_splitter import (
    RecursiveCharacterTextSplitter,
    MarkdownTextSplitter,
    TokenTextSplitter,
)

# ── Strategy 1: Recursive Character Splitting (Most Common) ──
def recursive_split(text: str, chunk_size: int = 1000, overlap: int = 200):
    """Split text using a hierarchy of separators."""
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=overlap,
        separators=["\n\n", "\n", ". ", ", ", " ", ""],
    )
    return splitter.split_text(text)

# ── Strategy 2: Semantic Chunking ──
def semantic_chunk(text: str, max_chunk_size: int = 1500):
    """Split text based on semantic similarity between sentences.

    This keeps semantically related content together, even if it means
    varying chunk sizes.
    """
    from sentence_transformers import SentenceTransformer
    import numpy as np

    model = SentenceTransformer("all-MiniLM-L6-v2")

    # Split into sentences first
    sentences = [s.strip() for s in text.split(". ") if s.strip()]
    if len(sentences) <= 1:
        return [text]

    # Get embeddings for each sentence
    embeddings = model.encode(sentences)

    # Calculate similarity between consecutive sentences
    similarities = []
    for i in range(len(embeddings) - 1):
        sim = np.dot(embeddings[i], embeddings[i + 1]) / (
            np.linalg.norm(embeddings[i]) * np.linalg.norm(embeddings[i + 1])
        )
        similarities.append(sim)

    # Find natural break points (low similarity = topic change)
    threshold = np.percentile(similarities, 30)  # Bottom 30% are break points

    chunks = []
    current_chunk = [sentences[0]]
    current_size = len(sentences[0])

    for i in range(1, len(sentences)):
        sentence = sentences[i]
        is_break = (i - 1 < len(similarities) and similarities[i - 1] < threshold)
        would_exceed = current_size + len(sentence) > max_chunk_size

        if is_break or would_exceed:
            chunks.append(". ".join(current_chunk) + ".")
            current_chunk = [sentence]
```

```
            current_size = len(sentence)
        else:
            current_chunk.append(sentence)
            current_size += len(sentence)

    if current_chunk:
        chunks.append(". ".join(current_chunk) + ".")

    return chunks

# ── Strategy 3: Parent-Child Chunking ──
def parent_child_chunks(text: str, parent_size: int = 2000, child_size: int = 400):
    """Create parent (large) and child (small) chunks.

    Child chunks are used for retrieval (more precise matching).
    Parent chunks are used for context (more complete information).
    When a child chunk matches, return its parent for the LLM.
    """
    # Create parent chunks
    parent_splitter = RecursiveCharacterTextSplitter(
        chunk_size=parent_size, chunk_overlap=200
    )
    parent_chunks = parent_splitter.split_text(text)

    # Create child chunks within each parent
    child_splitter = RecursiveCharacterTextSplitter(
        chunk_size=child_size, chunk_overlap=50
    )

    parent_child_map = []
    for parent_idx, parent in enumerate(parent_chunks):
        children = child_splitter.split_text(parent)
        for child in children:
            parent_child_map.append({
                "child_text": child,        # Used for retrieval
                "parent_text": parent,       # Used for LLM context
                "parent_index": parent_idx,
            })

    return parent_child_map
```

10.5 Evaluation Metrics for RAG Systems

Building reliable RAG systems requires systematic evaluation. Here are the key metrics and how to measure them:

```python
# src/rag_evaluation.py
"""Comprehensive RAG evaluation framework using the RAGAS approach."""

import json
from openai import OpenAI
from dotenv import load_dotenv
from dataclasses import dataclass

load_dotenv()
client = OpenAI()

@dataclass
class RAGEvalMetrics:
    """Standard RAG evaluation metrics."""
    faithfulness: float      # Is the answer grounded in the retrieved context?
    answer_relevancy: float  # Does the answer actually address the question?
    context_precision: float # Are the retrieved chunks relevant to the question?
    context_recall: float    # Did retrieval find all the needed information?

def evaluate_faithfulness(question: str, answer: str, context: str) -> float:
    """Score how well the answer is grounded in the provided context (0-1).

    Faithfulness measures whether every claim in the answer can be traced
    back to the retrieved context. High faithfulness = low hallucination.
    """
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{
            "role": "user",
            "content": f"""Evaluate faithfulness: Is the answer supported by the con-
text?

Context: {context}

Question: {question}

Answer: {answer}

Extract each claim from the answer and check if the context supports it.
Return JSON: {{"supported_claims": N, "total_claims": N, "score": 0.0-1.0,
"unsupported": ["claim1", ...]}}"""
        }],
        response_format={"type": "json_object"},
        temperature=0,
    )
    result = json.loads(response.choices[0].message.content)
    return result.get("score", 0.0)

def evaluate_answer_relevancy(question: str, answer: str) -> float:
    """Score how relevant the answer is to the question (0-1).

    A relevant answer directly addresses what was asked, without
    going off-topic or providing unrelated information.
    """
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{
            "role": "user",
            "content": f"""Score the relevancy of this answer to the question
(0.0-1.0).

Question: {question}
```

```
Answer: {answer}

Consider:
- Does the answer directly address the question?
- Is the answer complete?
- Is there unnecessary/off-topic information?

Return JSON: {{"score": 0.0-1.0, "explanation": "..."}}"""
        }],
        response_format={"type": "json_object"},
        temperature=0,
    )
    result = json.loads(response.choices[0].message.content)
    return result.get("score", 0.0)

def evaluate_context_precision(question: str, contexts: list[str]) -> float:
    """Score how relevant the retrieved contexts are to the question (0-1).

    High precision means most retrieved chunks are actually useful.
    Low precision means the retriever is pulling in irrelevant chunks.
    """
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{
            "role": "user",
            "content": f"""For each retrieved context chunk, score its relevance to
the question (0 or 1).

Question: {question}

Chunks:
{chr(10).join(f'[{i+1}] {c[:300]}' for i, c in enumerate(contexts))}

Return JSON: {{"relevant_chunks": [1, 0, 1, ...], "precision": 0.0-1.0}}"""
        }],
        response_format={"type": "json_object"},
        temperature=0,
    )
    result = json.loads(response.choices[0].message.content)
    return result.get("precision", 0.0)

def run_full_evaluation(
    rag_function,
    test_cases: list[dict],
) -> dict:
    """Run a full evaluation suite on a RAG system.

    Args:
        rag_function: Function that takes a question and returns
                      {"answer": str, "contexts": list[str]}
        test_cases: List of {"question": str, "ground_truth": str}
    """
    all_metrics = []

    for case in test_cases:
        result = rag_function(case["question"])
        answer = result["answer"]
        contexts = result.get("contexts", [])
        context_str = "\n\n".join(contexts)

        metrics = RAGEvalMetrics(
            faithfulness=evaluate_faithfulness(case["question"], answer, context_str),
            answer_relevancy=evaluate_answer_relevancy(case["question"], answer),
```

```
            context_precision=evaluate_context_precision(case["question"], contexts),
            context_recall=0.0,  # Requires ground truth context, simplified here
        )
        all_metrics.append(metrics)

    # Aggregate results
    n = len(all_metrics)
    return {
        "num_cases": n,
        "avg_faithfulness": round(sum(m.faithfulness for m in all_metrics) / n, 3),
        "avg_answer_relevancy": round(sum(m.answer_relevancy for m in all_metrics) /
n, 3),
        "avg_context_precision": round(sum(m.context_precision for m in all_metrics) /
n, 3),
        "overall_score": round(
            sum(
                (m.faithfulness + m.answer_relevancy + m.context_precision) / 3
                for m in all_metrics
            ) / n, 3
        ),
    }
```

Appendix A: Quick Reference — Common pixi.toml Configurations

```
# Minimal LLM project
[dependencies]
python = ">=3.11"
openai = ">=1.40"
python-dotenv = ">=1.0"

# RAG project
[dependencies]
python = ">=3.11"
langchain = ">=0.3"
langchain-openai = ">=0.2"
chromadb = ">=0.5"
pypdf = ">=4.0"

# Full agent project
[dependencies]
python = ">=3.11"
langchain = ">=0.3"
langgraph = ">=0.2"
langchain-openai = ">=0.2"
chromadb = ">=0.5"
tavily-python = ">=0.4"

# Local model project
[dependencies]
python = ">=3.11"
transformers = ">=4.44"
torch = ">=2.4"
accelerate = ">=0.33"
bitsandbytes = ">=0.43"
sentence-transformers = ">=3.0"
```

Appendix B: Glossary

Term	Definition
LLM	Large Language Model — a neural network trained on massive text data to understand and generate language
Token	The smallest unit of text processed by an LLM (roughly a word or subword)
Embedding	A dense numerical vector representing the semantic meaning of text
Context Window	The maximum number of tokens an LLM can process in a single request
Temperature	A parameter controlling the randomness/creativity of LLM outputs
RAG	Retrieval-Augmented Generation — combining information retrieval with LLM generation
Vector Database	A database optimized for storing and searching high-dimensional vectors
Cosine Similarity	A metric measuring the angle between two vectors; used for semantic similarity
Chunking	Splitting large documents into smaller pieces for embedding and retrieval
Agent	An AI system that autonomously reasons, plans, and uses tools to accomplish goals
ReAct	Reason + Act — an agent pattern combining reasoning with tool use
LangChain	A Python framework for building LLM-powered applications and agents
LlamaIndex	A Python framework specialized in data indexing and retrieval for RAG
LangGraph	An extension of LangChain for building stateful, graph-based agent workflows
Quantization	Reducing model precision (e.g., 32-bit to 4-bit) to decrease memory requirements
Fine-tuning	

Term	Definition
	Further training a pre-trained model on domain-specific data
Prompt Engineering	The art of crafting effective prompts to get optimal outputs from LLMs
Hallucination	When an LLM generates plausible-sounding but factually incorrect information
LCEL	LangChain Expression Language — a declarative way to compose LangChain components

Appendix C: Recommended Learning Path

```
Week 1-2:  Sections 1-2 — Environment setup, first API calls
Week 3-4:  Section 3 — Deep understanding of LLM fundamentals
Week 5-6:  Section 4 — Master API usage across providers
Week 7-8:  Section 5 — Local models and HuggingFace
Week 9-12: Section 6 — RAG from basic to advanced
Week 13-16: Section 7 — Agent patterns and frameworks
Week 17-18: Section 8 — Advanced topics and best practices
Week 19-20: Section 9 — Build the capstone project
```

This guide was created as a comprehensive learning resource for mastering Generative AI, RAG, and Agentic AI in Python. All code examples are designed to be copy-paste runnable with the provided pixi.toml configurations. For the latest updates and corrections, consult the official documentation of each framework and API provider.